
OGRe: An Object-Oriented General Relativity Package for Mathematica

By **Barak Shoshany**

Email: baraksh@gmail.com

Website: <https://baraksh.com/>

GitHub: <https://github.com/bshoshany>

Project GitHub repository: <https://github.com/bshoshany/OGRe>

This is the complete documentation for **v2.0.0** of the package, released on **2026-05-02**.

Introduction

Overview

OGRe is a modern Mathematica package for differential geometry and tensor calculus, designed to be both powerful and user-friendly. It can be used in a variety of contexts where tensor calculations are needed, in both mathematics and physics, but it is especially suitable for general relativity.

Tensors are abstract objects, which can be represented as multi-dimensional arrays once a choice of index configuration and coordinate system is made. OGRe stays true to this definition, but takes away the complexities that come with combining tensors in different representations. This is done using an object-oriented programming approach, taking advantage of principles such as encapsulation and class invariants to eliminate the possibility of user error.

The user initially defines each tensor in OGRe using its explicit components in any single representation. Operations on this tensor are then done abstractly, without needing to specify which representation to use. Possible operations include addition of tensors, multiplication of a tensor by a scalar, trace, contraction, and partial and covariant derivatives.

OGRe will automatically choose which representation to use for each tensor based on how the tensors are combined. For example, if two tensors are added, then OGRe will automatically use the same index configuration for both. Similarly, if two tensors are contracted, then OGRe will automatically ensure that the contracted indices are one upper (contravariant) and one lower (covariant). OGRe will also automatically transform all tensors being operated on to the same coordinate system.

Transformations between representations are done behind the scenes; all the user has to do is specify which metric to use for raising and lowering indices, and how to transform between the coordinate

systems being used. This also means that there is no room for user error. The user cannot mistakenly perform "illegal" operations such as $2 A^{\mu\nu} + B_{\mu\lambda} C_{\lambda\nu}$. Instead, the user simply inputs the names of the tensors, the order (but **not** the configuration) of indices for each, and the operations to perform - and the correct combination $2 A^{\mu\nu} B^\mu{}_\lambda C^{\lambda\nu}$ will be automatically deduced.

I initially created OGRE for use in my own research, so I made it as flexible and powerful as possible. I also wanted my students to be able to use it easily and efficiently, even if they only have minimal experience with Mathematica and/or general relativity, so I made it simple to learn and easy to use. As a result, this package is equally suitable for both experienced and novice researchers.

Features

- Define coordinate systems and the transformation rules between them. Tensor components are then transformed automatically between coordinates behind the scenes as needed.
- Each tensor is associated with a specific metric. Tensor components are then transformed automatically between different index configurations, raising and lowering indices behind the scenes as needed.
- Get the components of any tensor in any index configuration and coordinate system. Display or generate TeX code for the components, either in vector/matrix form or as a list of all unique non-zero elements. Metrics can also be displayed as a line element. Arbitrary functions or replacement rules can easily be applied to the components before they are displayed.
- Automatically simplify tensor components, optionally with user-defined simplification functions and/or assumptions. Simplifications can be parallelized for a significant performance boost. The user may utilize the package's simplification algorithm for any Mathematica expression, not just tensors.
- Export tensors to a Mathematica notebook or to a file, so they can later be imported into another Mathematica session without having to redefine them from scratch.
- Highly customizable. User settings are exported and imported along with the tensors. Some settings are persistent between sessions.
- Easily calculate arbitrary tensor formulas using any combination of addition, multiplication by a scalar, trace, contraction, partial derivative, and covariant derivative.
- Built-in modules for calculating the Christoffel symbols (Levi-Civita connection), Riemann tensor, Ricci tensor and scalar, Kretschmann scalar, Einstein tensor, Weyl tensor, curve Lagrangian, and volume element from a metric.
- Calculate the norm-squared of tensors of any rank, and the determinant and volume element of any metric.
- Calculate the geodesic equations in terms of a user-defined affine curve parameter, in two different ways: from the Christoffel symbols or from the curve Lagrangian. For spacetime metrics, the geodesic equations can be calculated in terms of the time coordinate.
- Designed with speed and performance in mind, using optimized algorithms designed specifically for this package. Derived tensors are cached on first use to avoid recalculating.

- Tensor IDs are auto-completed in the notebook interface when used with any suitable ORe module, making the interface more user-friendly and easy to use.
- Fully portable. Can be imported directly from the web into any Mathematica notebook, without downloading or installing anything. Integrates seamlessly with the Wolfram Cloud.
- Clear and detailed documentation, with many examples, in both Mathematica notebook and PDF format. The notebook `ORe_Documentation.nb` can also be opened with the free Wolfram Player. Detailed usage messages are also provided.
- Open source. The code is extensively documented; please feel free to fork and modify it as you see fit.
- Under continuous and active development. Bug reports and feature requests are welcome, and should be made via GitHub issues.
- Thoroughly tested. A comprehensive internal test suite thoroughly tests all ORe modules and verifies that it accurately reproduces a wide variety of known results from differential geometry and general relativity, ensuring users can fully trust ORe as a reliable tool for research and pedagogy.

The object-oriented design philosophy

Object-oriented programming refers to a paradigm where a program's code is organized around objects. An **object** belongs to a user-defined type, called a **class**. The class defines the **data** that the object stores, as well as **methods** or **member functions** that read or manipulate that data. One of the fundamental principles of object-oriented programming is **encapsulation**, which means that the user may only access an object's data using the methods defined by the class, and is unable to access the object's data directly.

Importantly, encapsulation allows for the preservation of **class invariants**. An invariant is a condition of validity that can always be assumed to be satisfied by the data stored in each object. If the methods make sure to preserve the invariant whenever they store or manipulate the data, and the user is prevented from changing the data manually and thus potentially violating the invariant, then the implementation of the class can be greatly simplified, and performance can be improved, because we will not need to verify that the data is valid every time we perform an operation.

The main idea behind ORe is to simplify the use of tensors by encoding all the information about a tensor in a single, self-contained object. As we mentioned above, a tensor is an abstract object. We can find components which represent this abstract entity in a particular coordinate system and index configuration, but the tensor is **not** its components. In ORe, a tensor object is initially defined (or **constructed**) by providing the components of the tensor in a particular representation - but once this is done, the user does not need to worry about coordinates or indices anymore, or even remember which coordinates and indices were initially used. The abstract tensor object will automatically transform the initial data to a different coordinate system or index configuration as needed, based on the context in which it was used.

As a tensor object holds the components of the same tensor in many different representations, the most important class invariant is the assumption that the different components indeed represent the

same tensor. This is achieved using encapsulation; the object's data can only be modified by private methods that preserve the invariant, and thus the user cannot accidentally cause a violation of the invariant by assigning components to one representation that are not related to the components of all other representations by the appropriate coordinate and/or index transformation.

Unfortunately, Mathematica does not have built-in support for object-oriented programming. However, version 10.0 of Mathematica, released in 2014, introduced the **Association** symbol. An **Association** is an **associative array**; it is similar to a **List**, except that instead of being just an array of values, an **Association** is a list of keys with a value associated to each key. This allows us to easily implement a rudimentary form of object-oriented programming, storing the properties of each object in the keys of a corresponding **Association**.

Of course, as Mathematica is not truly object-oriented, there is no actual "tensor class" defined anywhere in the package. Instead, the tensor class exists only **implicitly**, as a design paradigm. Furthermore, the functions that process the data stored in the tensor objects are not methods of a class, they are simply modules that take tensor objects as input and/or produce tensor objects as outputs. (In earlier versions, I tried using a syntax that resembled method syntax in languages such as C++ or Python, but eventually decided against it, as it was too cumbersome.) Still, designing the package with object-oriented programming in mind allows us to reap many of this paradigm's benefits, as explained above - and it simply makes sense for tensors, due to their abstract and multifaceted nature.

Installing and loading the package

This package is compatible with Mathematica 14.0 or later. It consists of only one file, **OGRe.m**. There are several different ways to load the package:

- **Run from local file with installation:** This is the recommended option, as it allows you to permanently use the package offline from any Mathematica notebook. Download the file **OGRe.m** from the GitHub repository and copy it to the directory given by `FileNameJoin[{$UserBaseDirectory, "Applications"}]`. The package may now be loaded from any notebook by writing `Needs["OGRe`"]` (note the backtick ` following the word OGRE).
- **Run from local file without installation:** This option allows you to use the package in a portable fashion, without installing it in the **Applications** directory. Download the file **OGRe.m** from the GitHub repository, place it in the same directory as the notebook you would like to use, and use the command `Get["OGRe.m", Path->NotebookDirectory[]]` to load the package.
- **Run from web with installation:** This option allows you to automatically download and install the package on any computer. Simply run the command `URLDownload["https://raw.githubusercontent.com/bshoshany/OGRe/master/OGRe.m", FileNameJoin[{$UserBaseDirectory, "Applications", "OGRe.m"}]]` from any Mathematica notebook to permanently install the package. Then use `Needs["OGRe`"]` from any notebook to load it.
- **Run from web without installation:** This option allows you to use the package from any Mathematica notebook on any computer, without manually downloading or installing it, as long as you have a working Internet connection. It also ensures that you always use the latest version of the

package, but be aware that updates may sometimes not be fully backwards compatible. Simply write `Get["https://raw.githubusercontent.com/bshoshany/OGRe/master/OGRe.m"]` in any Mathematica notebook to load the package.

Note: In the "run from web" options, if you wish to load a specific version for compatibility reasons, replace `master` in the URL with the full version number including the `v` prefix, e.g.

`https://raw.githubusercontent.com/bshoshany/OGRe/v2.0.0/OGRe.m` for v2.0.0.

To **uninstall** the package, just delete the file from the `Applications` directory, which can be done from within Mathematica using the command

`DeleteFile[FileNameJoin[{UserBaseDirectory, "Applications", "OGRe.m"}]]`.

Getting started

For the purposes of this documentation, I will use the "run from local file without installation" option, since you most likely downloaded both the documentation and the package together:

```
In[ ]:= Get["OGRe.m", Path → NotebookDirectory[]]
```

OGRe: An Object-Oriented General Relativity Package for Mathematica
By Barak Shoshany (baraksh@gmail.com) (baraksh.com)
v2.0.0 (2026-05-02)
GitHub repository: <https://github.com/bshoshany/OGRe>
Release notes: [view on GitHub](#) | [view in this notebook](#)

OGRe:

- To view the full documentation for the package, type `TDocs[]`.
- To list all available modules, type `?OGRe`*``.
- To get help on a particular module, type `?`` followed by the module name.
- To enable parallelization, type `TSetParallelization[True]`.
- To check for updates, type `TCheckForUpdates[]`.
- To enable automatic checks for updates at startup, type `TSetAutoUpdates[True]`.

The package displays a welcome message upon loading, which provides some information on how to get started. As stated in the welcome message, to list all of the modules available in this package, you may use the following command:

In[]:=

? OGRE` *

Out[]=

▼ OGRE`			
TAddCoordTransformati` on	TExport	TMetricDeterminant	TSetExactSignChecks
TCalc	TExportAll	TNewCoordinates	TSetIndexLetters
TChangeDefaultCoords	TGeodesicFromChristoff` el	TNewMetric	TSetParallelization
TChangeDefaultIndices	TGeodesicFromLagrangi` an	TNewTensor	TSetReservedSymbols
TChangeID	TGeodesicWithTimePar` ameter	TNormSquared	TSetSimplifyFunc
TChangeSymbol	TGetComponents	TPartialD	TShow
TCheckForUpdates	TImport	TRank	TSimplify
TChristoffel	TImportAll	TRicci	TTexList
TCite	TInfo	TRicciScalar	TTexShow
TCovariantD	TKretschmann	TRiemann	TVolumeElement
TDelete	TLagrangian	TSetAllowOverwrite	TWeyl
TDim	TLineElement	TSetAssumptions	\$CurveParam
TDocs	TList	TSetAutoUpdates	
TEinstein	TMessage	TSetCurveParameter	

Clicking on the name of any module in this list will show its usage message. Notice that all OGRE modules start with the letter **T**, to help distinguish them from other modules, whether built-in or from other packages.

When the package loads, it will automatically check the GitHub repository for updates. This can also be done manually using the module **TCheckForUpdates**. If a new version of the package has been released, you will be given the option to download or install it in one click. If you are running the package directly from the web, you will always be using the latest version.

In[]:=

? TCheckForUpdates

Out[]=

Symbol
TCheckForUpdates[] checks the GitHub repository for new versions of this package. If a new version is available, the user will be given the option to download or install it.
▼

You can also disable automatic checks for updates at startup in all future sessions by calling **TSetAutoUpdates[False]**:

In[]:=
Out[]:=

? TSetAutoUpdates

Symbol

TSetAutoUpdates[**False**] turns off automatic checks for updates at startup. TSetAutoUpdates[**True**] turns them back on, which is the default setting. TSetAutoUpdates[] returns the current setting. Note that this setting is persistent between sessions.

If at any point you wish to view this documentation, simply type `TDocs[]`. The file `OGrE_Documentation.nb` will then be automatically opened in Mathematica.

In[]:=
Out[]:=

? TDocs

Symbol

TDocs[] opens the Mathematica notebook `OGrE_Documentation.nb` from the GitHub repository, which contains the full documentation for the package.

Creating and displaying tensor objects

Defining coordinates

To define tensors, we first need to define the manifold on which they reside. Since we are focusing on general relativity, we will use 4-dimensional spacetime manifolds in the following examples, but this package works equally well with manifolds that are purely spatial and/or have a different number of dimensions.

The first step is to define the coordinate system. In OGrE, coordinates are represented as a special tensor object: a vector x^μ (a tensor of rank 1) representing a point. To define the coordinates, we use the module `TNewCoordinates`:

In[]:=
Out[]=

? TNewCoordinates

Symbol

TNewCoordinates[**coordinatesID**, **symbols**], creates a new tensor object representing a coordinate system.
coordinatesID is a string that will be used to identify the new object, and must be unique.
symbols are the coordinate symbols, e.g. {t, x, y, z}. They will automatically be cleared and protected against future changes using TSetReservedSymbols[].

For example, to define the **Cartesian coordinate system**, we use the following syntax:

In[]:=
Out[]=

TNewCoordinates["Cartesian", {t, x, y, z}]

Cartesian

The first argument is the new tensor object's **unique ID**. This is the string that we will use to refer to this tensor from now on, and it will also be displayed whenever we print the tensor. The ID string is case-sensitive, can include any characters, and can be of any length, but it is recommended to keep it short and simple. Once a tensor object is created, you cannot create another tensor object with the same ID unless you rename or delete it first (see below). Note that the ID string is also the return value of the module; generally, all modules that operate on a tensor object will return its ID string as output. This allows us to compose different modules together, as we will see below.

The second argument is the list of coordinates. Note that the order of coordinates matters, as it will determine the order of components in the tensors defined on this manifold. The symbols used also matter, as tensor components will usually be functions of these symbols. We can similarly define the spherical coordinate system:

In[]:=
Out[]=

TNewCoordinates["Spherical", {t, r, θ , ϕ }]

Spherical

It is crucial that the coordinate symbols, in this case **t**, **r**, **θ** , **ϕ** , remain as **undefined** symbols throughout the calculation, otherwise errors may occur. For example, if our metric contains functions of **r**, and at some point in the notebook we set **r = 1**, then Mathematica will replace every instance of **r** with **1**, which means those abstract functions will be replaced with their values evaluated at **r = 1**. Furthermore, if we, for example, want to take the derivative with respect to **r** (e.g. for the purpose of calculating various curvature tensors), this will not be possible, since one cannot take a derivative with respect to a number.

To prevent such errors, **TNewCoordinates** automatically clears any previous definitions of the given symbols, and also protects them against future changes. This is done using the **TSetReservedSymbols**

module:

In[*n*]:=

? TSetReservedSymbols

Out[*n*]=

Symbol

TSetReservedSymbols[**symbol**] clears any definitions previously used for **symbol** and protects it against future changes. **symbol** can be either a symbol name or a string representing a symbol name. After completion, the module will return a list of the currently reserved symbols.

Useful for making sure coordinate variables, parameters, and abstract functions used in tensors remain abstract symbols and do not accidentally obtain values and break the code.

If the reserved symbol is a function, this function will be displayed without arguments when using TList[] and TShow[], for improved readability. The reserved symbols will be exported when using TExportAll[] so they can later be imported using TImportAll[].

TSetReservedSymbols[{**symbol1**, **symbol2**, ...}] reserves all of the given symbols. Each of the symbols can be either a symbol name or a string representing a symbol name.

TSetReservedSymbols[] returns the currently reserved symbols.

Indeed, if we now try to give **r** the value **1**, we will get an error:

In[*n*]:=

r = 1

Out[*n*]=

Set: Symbol r is Protected. ⓘ

1

You can also use **TSetReservedSymbols** manually for any constants or parameters used in the tensor, as we will demonstrate in the next section. Note that it is always possible to replace reserved symbols with numbers or other expressions using /. or **ReplaceAll**, so giving a value to a reserved symbol directly is not something you will ever need to do.

Defining metrics

To finish defining a manifold, we need to define its metric tensor. For this, we use the module **TNewMetric**:

In[]:=
Out[]:=

? TNewMetric

Symbol

TNewMetric[**metricID**, **coordinatesID**, **components**, **symbol**] creates a new tensor object representing a metric.

metricID is a string that will be used to identify the new object, and must be unique.

coordinatesID is the unique ID of a tensor object representing a coordinate system, created using TNewCoordinates[].

components is a square, symmetric, and invertible matrix representing the metric with two lower indices in that coordinate system.

symbol will be used to represent the metric in formulas. If not given, "g" will be used.

Again, we must input a unique ID for the new tensor object corresponding to the metric. We also input the unique ID of a coordinate system created using **TNewCoordinates**. This coordinate system is the one in which the components of the metric are initially given, but they will be automatically transformed to other coordinate systems later as needed. Note that the components are assumed to be the representation of the metric with two lower indices, since that is how metrics are usually defined; one upper and one lower index is just the identity matrix, and two upper indices is the inverse metric. Optionally, we can also specify a symbol to be used for representing the metric.

Let us use this module to create a tensor object for the **Minkowski metric**, specifying the components in Cartesian coordinates:

In[]:=
Out[]:=

TNewMetric["Minkowski", "Cartesian", DiagonalMatrix[{-1, 1, 1, 1}], "η"]

Minkowski

As with **TNewCoordinates**, we received the ID string for the new tensor object as output.

Similarly, let us define the **Schwarzschild metric**, this time specifying the components in spherical coordinates. However, before we can safely do so, we should take one additional step. When we defined the coordinates using **TNewCoordinates**, the symbols used were automatically reserved, that is, cleared and protected from future changes, using **TSetReservedSymbols**. However, **TNewMetric** does not automatically reserve any symbols it finds in the metric, since you might not want to reserve some of them. Since the Schwarzschild metric has a free parameter **M**, the mass of the black hole, we must reserve this symbol manually:

```
In[ ]:= TSetReservedSymbols [M]
Out[ ]=
```

```
{t, x, y, z, r, θ, φ, M}
```

Note that **TSetReservedSymbols** returned the full list of symbols we reserved so far: the coordinate symbols and the black hole's mass.

Now we can define the Schwarzschild metric:

```
In[ ]:= TNewMetric["Schwarzschild", "Spherical",
  DiagonalMatrix[{- (1 - 2 M / r), 1 / (1 - 2 M / r), r^2, r^2 Sin[θ]^2}]]
Out[ ]=
```

```
Schwarzschild
```

Note that since we did not specify a symbol, the symbol **g** will be used by default, as demonstrated below.

Displaying tensors

OGRe contains two modules for displaying the contents of tensors. The first one is **TShow**, which shows the ID, symbol, indices, coordinates, and components in those indices and coordinates, in vector or matrix form when applicable:

```
In[ ]:= ? TShow
Out[ ]=
```

Symbol

TShow[ID] shows the components of the tensor object **ID** in its default index configuration and coordinate system.

TShow[ID, indices] shows the components in the index configuration **indices**, which should be a list of the form $\{\pm 1, \pm 1, \dots\}$, where +1 corresponds to an upper index and -1 corresponds to a lower index.

TShow[ID, coordinatesID] shows the components in the coordinate system **coordinatesID**.

TShow[ID, rules] applies **ReplaceAll[rules]** to each of the tensor's elements, and then automatically simplifies them, before they are displayed. **rules** can be either a single rule of the form **lhs** $\rightarrow$ **rhs** or a list of rules.

TShow[ID, function] maps **function** to each of the tensor's elements, and then automatically simplifies them, before they are displayed.

TShow[ID, indices, coordinatesID, rules, function] does all of the above; any of the arguments can be omitted. If both **rules** and **function** are given, the rules will be applied before the function.

Coordinates are also tensor objects, so we can use **TShow** to show the two coordinate tensors we defined above:

In[]:= **TShow["Cartesian"]**

OGRe:

$$\text{Cartesian: } x^\mu = \begin{pmatrix} t \\ x \\ y \\ z \end{pmatrix}$$

In[]:= **TShow["Spherical"]**

OGRe:

$$\text{Spherical: } x^\mu = \begin{pmatrix} t \\ r \\ \theta \\ \phi \end{pmatrix}$$

Note that coordinate tensors always have the symbol x .

We can also show the two metrics we created using these coordinates:

In[]:= **TShow["Minkowski"]**

OGRe:

$$\text{Minkowski: } \eta_{\mu\nu}(t, x, y, z) = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

In[]:= **TShow["Schwarzschild"]**

OGRe:

$$\text{Schwarzschild: } g_{\mu\nu}(t, r, \theta, \phi) = \begin{pmatrix} -1 + \frac{2M}{r} & 0 & 0 & 0 \\ 0 & \frac{1}{1 - \frac{2M}{r}} & 0 & 0 \\ 0 & 0 & r^2 & 0 \\ 0 & 0 & 0 & r^2 \sin^2[\theta] \end{pmatrix}$$

The other module available in OGRE for displaying the contents of tensors is **TList**, which lists all of the **unique (up to sign), non-zero** components of the tensor. It is usually the best option for higher-rank tensors, which cannot be displayed in vector or matrix form, such as the Christoffel symbols or Riemann tensor (see below). Its syntax is:

In[]:=
Out[]=

? TList

Symbol

TList[ID] lists the unique, non-zero components of the tensor object **ID** in its default index configuration and coordinate system.

TList[ID, indices] lists the components in the index configuration **indices**, which should be a list of the form {±1, ±1, ...}, where +1 corresponds to an upper index and -1 corresponds to a lower index.

TList[ID, coordinatesID] lists the components in the coordinate system **coordinatesID**.

TList[ID, rules] applies ReplaceAll[rules] to each of the tensor's elements, and then automatically simplifies them, before they are displayed. **rules** can be either a single rule of the form **lhs** → **rhs** or a list of rules.

TList[ID, function] maps **function** to each of the tensor's elements, and then automatically simplifies them, before they are displayed.

TList[ID, indices, coordinatesID, rules, function] does all of the above; any of the arguments can be omitted.

If both **rules** and **function** are given, the rules will be applied before the function.

Use TSetExactSignChecks to enable or disable exact sign checks. When exact sign checks are enabled (the default), if TList cannot detect equality of two components up to sign by direct comparison, it checks whether the sum of the components can be proven to equal zero, which may require simplification. For tensors with complicated components, this may take a long time, in which case this option can be disabled; however, TList will then only be able to detect when two components are the same up to sign in simple cases.

For example:

In[]:=

TList["Minkowski"]

OGRe:

Minkowski:

$$\eta_{tt} = -\eta_{xx} = -\eta_{yy} = -\eta_{zz} = -1$$

In[]:=

TList["Schwarzschild"]

OGRe:

Schwarzschild:

$$g_{tt} = -1 + \frac{2M}{r}$$

$$g_{rr} = \frac{1}{1 - \frac{2M}{r}}$$

$$g_{\theta\theta} = r^2$$

$$g_{\phi\phi} = r^2 \sin[\theta]^2$$

Note that both TShow and TList display their outputs using the DisplayFormula Notebook style. It's up to the user to decide how to define this style; in this notebook, I used a font size of 20, aligned to

center. The style may be easily changed by clicking on the "Format" menu in Mathematica and selecting "Edit Stylesheet". Then, choose the **DisplayFormula** style, select that cell, and modify its format using the "Format" menu.

If, as in the examples above, no additional arguments are given to **TShow** and **TList**, they display the tensors in their **default indices** and **default coordinates**, which are the ones first used to define the tensor (unless you change them later). So, for example, the default indices of the Minkowski metric are two lower indices, and its default coordinates are Cartesian. We will show later how to change these defaults, and how to display any tensor in any index configuration and coordinate system.

By default, **TList** uses **exact sign checks** to detect when two components are the same up to a sign. This is controlled by **TSetExactSignChecks []**:

In[]:=
Out[]:=

? TSetExactSignChecks

Symbol

TSetExactSignChecks[] shows whether exact sign checks are enabled.

TSetExactSignChecks[**True**] enables exact sign checks, and TSetExactSignChecks[**False**] disables them.

TSetExactSignChecks[Automatic] resets exact sign checks to the default setting, which is **True**.

When exact sign checks are enabled, if TList cannot detect equality of two components up to sign by direct comparison, it checks whether the sum of the components can be proven to equal zero. It first tries a fast exact algebraic zero test, and if that fails, it falls back to the simplification function configured with TSetSimplifyFunc. For tensors with complicated components, this may take a long time, in which case this option can be disabled; however, TList will then only be able to detect when two components are the same up to sign in simple cases.

When exact sign checks are enabled, if **TList** cannot detect equality of two components up to sign by direct comparison, it checks whether the sum of the components can be proven to equal zero. It first tries a fast exact algebraic zero test, and if that fails, it falls back to the configured simplification function (see below). For tensors with complicated components, this may take a long time, in which case this option can be disabled using **TSetExactSignChecks [False]**; however, **TList** will then only be able to detect when two components are the same up to sign in simple cases.

Often, we might want to copy the output of **TShow** or **TList** into a LaTeX document. Copying the output cells directly in Mathematica, or even using "Copy As > LaTeX", usually won't produce good results, due to the way these modules structure the notebook output so it is displayed nicely. Instead, the modules **TTexShow** and **TTexList** accept the same arguments as **TShow** and **TList**, respectively, but return the formatted output as a TeX string instead of displaying it in the notebook. For example:

In[]:=

? TTeXShow

Out[]:=

Symbol

TTeXShow[**ID**] returns a TeX string for the components of the tensor object **ID** in its default index configuration and coordinate system.

TTeXShow[**ID**, **indices**] returns the components in the index configuration **indices**, which should be a list of the form $\{\pm 1, \pm 1, \dots\}$, where +1 corresponds to an upper index and -1 corresponds to a lower index.

TTeXShow[**ID**, **coordinatesID**] returns the components in the coordinate system **coordinatesID**.

TTeXShow[**ID**, **rules**] applies ReplaceAll[**rules**] to each of the tensor's elements, and then automatically simplifies them, before they are converted to TeX. **rules** can be either a single rule of the form **lhs** $\rightarrow$ **rhs** or a list of rules.

TTeXShow[**ID**, **function**] maps **function** to each of the tensor's elements, and then automatically simplifies them, before they are converted to TeX.

TTeXShow[**ID**, **indices**, **coordinatesID**, **rules**, **function**] does all of the above; any of the arguments can be omitted. If both **rules** and **function** are given, the rules will be applied before the function.

Note: It is recommended to use CopyToClipboard@TTeXShow[...] to copy the resulting TeX string to the clipboard directly.



In[]:=
Out[]:=

? TTeXList

Symbol

TTeXList[ID] returns a TeX string for the unique, non-zero components of the tensor object **ID** in its default index configuration and coordinate system.

TTeXList[ID, **indices**] returns the components in the index configuration **indices**, which should be a list of the form {±1, ±1, ...}, where +1 corresponds to an upper index and -1 corresponds to a lower index.

TTeXList[ID, **coordinatesID**] returns the components in the coordinate system **coordinatesID**.

TTeXList[ID, **rules**] applies ReplaceAll[**rules**] to each of the tensor's elements, and then automatically simplifies them, before they are converted to TeX. **rules** can be either a single rule of the form **lhs** → **rhs** or a list of rules.

TTeXList[ID, **function**] maps **function** to each of the tensor's elements, and then automatically simplifies them, before they are converted to TeX.

TTeXList[ID, **indices**, **coordinatesID**, **rules**, **function**] does all of the above; any of the arguments can be omitted. If both **rules** and **function** are given, the rules will be applied before the function.

Use TSetExactSignChecks to enable or disable exact sign checks. When exact sign checks are enabled (the default), if TTeXList cannot detect equality of two components up to sign by direct comparison, it checks whether the sum of the components can be proven to equal zero, which may require simplification. For tensors with complicated components, this may take a long time, in which case this option can be disabled; however, TTeXList will then only be able to detect when two components are the same up to sign in simple cases.

Note: It is recommended to use CopyToClipboard@TTeXList[...] to copy the resulting TeX string to the clipboard directly.

Note that TTeXShow and TTeXList return the TeX output as a string. If we run them in an input cell, the string will be placed in an output cell, and if we just select that string and copy it, Mathematica will usually mess up the formatting, resulting in invalid TeX code. Instead, the best way to copy the TeX output is to use CopyToClipboard to copy it directly to the clipboard, for example:

In[]:= CopyToClipboard@TTeXShow["Schwarzschild"]

Line and volume elements

The **line element** for a metric $g_{\mu\nu}$ is defined as

$$ds^2 = g_{\mu\nu} dx^\mu dx^\nu.$$

In OGRe, we can calculate the line element using the module **TLineElement**:

```
In[*]:= ? TLineElement
Out[*]=
```

Symbol

TLineElement[**metricID**] returns the line element of the metric **metricID** in its default coordinate system.

TLineElement[**metricID**, **coordinatesID**] returns the line element in the coordinate system **coordinatesID**.

▼

For example:

```
In[*]:= TLineElement["Minkowski"]
Out[*]=
```

$$-dt^2 + dx^2 + dy^2 + dz^2$$

```
In[*]:= TLineElement["Schwarzschild"]
Out[*]=
```

$$\frac{dr^2}{1 - \frac{2M}{r}} + \left(-1 + \frac{2M}{r}\right) dt^2 + r^2 d\theta^2 + r^2 \sin^2[\theta] d\phi^2$$

Note that these are standard Mathematica expressions, so they can be manipulated like any other expressions, including operations such as simplifying or factoring. As an example of a more interesting (non-diagonal) line element, consider the **Alcubierre warp drive metric**:

```
In[*]:= TSetReservedSymbols[{v, f}];
TNewMetric["Alcubierre", "Cartesian",
  {{-1 + v[t]^2 f[t, x, y, z]^2, 0, 0, -v[t] × f[t, x, y, z]},
   {0, 1, 0, 0}, {0, 0, 1, 0}, {-v[t] × f[t, x, y, z], 0, 0, 1}}];
```

This is what the metric looks like in matrix form:

```
In[*]:= TShow["Alcubierre"]
OGRe:
```

Alcubierre: $g_{\mu\nu}(t, x, y, z) = \begin{pmatrix} -1 + f^2 v^2 & 0 & 0 & -f v \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -f v & 0 & 0 & 1 \end{pmatrix}$

f is a **form function** which is equal to 1 inside a "warp bubble" of finite radius and 0 outside it, and v is the velocity of the bubble, which can be faster than the speed of light ($v > 1$). Note that we reserved both v and f , since they are parameters used in the metric. Here we see another advantage of reserving symbols: since v and f are reserved symbols, and they are functions of the coordinates only, their

arguments are not shown when using `TShow` and `TList`, for improved readability.

It is easy to see that the metric is flat where $f = 0$, that is, outside the bubble. Its line element is:

```
In[ ]:= lineElement = TLineElement["Alcubierre"]
Out[ ]:= -dt^2 + dx^2 + dy^2 + dz^2 + dt f[t, x, y, z] × v[t] (-2 dz + dt f[t, x, y, z] × v[t])
```

We can simplify it by doing some clever factorization:

```
In[ ]:= lineElement =
lineElement // Expand // (#1[[1]] + #1[[2]] + #1[[3]] + Factor[#1[[4]] + #1[[5]] + #1[[6]]) &
Out[ ]:= -dt^2 + dx^2 + dy^2 + (dz - dt f[t, x, y, z] × v[t])^2
```

In this form, it is immediately clear that the metric is flat outside the warp bubble (where f is 0), and inside the warp bubble (when f is 1) it is a flat metric translated by an amount $v(t) dt$ in the z direction:

```
In[ ]:= lineElement /. f -> (0 &)
Out[ ]:= -dt^2 + dx^2 + dy^2 + dz^2
```

```
In[ ]:= lineElement /. f -> (1 &)
Out[ ]:= -dt^2 + dx^2 + dy^2 + (dz - dt v[t])^2
```

Another thing we can do with a metric is calculate its **volume element**, defined as

$$dV = \sqrt{|\det(g)|} dx^0 \wedge \dots \wedge dx^{n-1},$$

where n is the number of dimensions. This can be done using the module `TVolumeElement`:

```
In[ ]:= ? TVolumeElement
Out[ ]:=
```

Symbol

`TVolumeElement[metricID]` returns the volume element of the metric **metricID** in its default coordinate system.

`TVolumeElement[metricID, coordinatesID]` returns the volume element in the coordinate system **coordinatesID**.

▼

For example:

```
In[ ]:= TVolumeElement["Minkowski"]
```

```
Out[ ]:=  $dt \wedge dx \wedge dy \wedge dz$ 
```

```
In[ ]:= TVolumeElement["Schwarzschild"]
```

```
Out[ ]:=  $r^2 \text{Abs}[\text{Sin}[\theta]] dt \wedge dr \wedge d\theta \wedge d\phi$ 
```

```
In[ ]:= TVolumeElement["Alcubierre"]
```

```
Out[ ]:=  $dt \wedge dx \wedge dy \wedge dz$ 
```

Note that the volume element involves the square root of the absolute value of the determinant of the metric. If we want the determinant itself, we can use **TMetricDeterminant**:

```
In[ ]:= ? TMetricDeterminant
```

```
Out[ ]:=
```

Symbol

TMetricDeterminant[**metricID**] returns the determinant of the metric **metricID** in its default coordinate system.

TMetricDeterminant[**metricID**, **coordinatesID**] returns the determinant in the coordinate system **coordinatesID**.

▼

For example:

```
In[ ]:= TMetricDeterminant["Schwarzschild"]
```

```
Out[ ]:=  $-r^4 \text{Sin}[\theta]^2$ 
```

Choosing index letters

By default, the **TShow** module uses Greek letters for the indices, in a specific order. The letters can be displayed or changed using the **TSetIndexLetters** module:

In[]:=
Out[]:=

? TSetIndexLetters

Symbol

TSetIndexLetters[] shows the index letters used when displaying indices.

TSetIndexLetters[**letters**] changes the index letters.

TSetIndexLetters[Automatic] resets the index letters to the default: "μνρσκλαβγδεζηθιξπτφχψω".

The default letters are:

In[]:=
Out[]:=

TSetIndexLetters []

μνρσκλαβγδεζηθιξπτφχψω

This means that the letter μ will be used for the first index, ν for the second, and so on. However, sometimes we want to use different letters. For example, let us change the indices to lowercase English letters:

In[]:=
Out[]:=

TSetIndexLetters ["abcdefghijklmnopqrstuvwxyz"]

abcdefghijklmnopqrstuvwxyz

TShow will now use these letters - in this particular order - when displaying tensors:

In[]:=

TShow["Minkowski"]

OGRe:

$$\text{Minkowski: } \eta_{ab}(t, x, y, z) = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Finally, let us reset the letters to the default:

In[]:=
Out[]:=

TSetIndexLetters [Automatic]

μνρσκλαβγδεζηθιξπτφχψω

Note that **TList** always uses the coordinate symbols themselves for the indices (e.g. $\eta_{t\,t}$, $\eta_{x\,x}$, etc.), so it is not affected by **TSetIndexLetters**.

Creating tensors in a given manifold

Any tensors other than coordinates and metrics are created using the module **TNewTensor**:

In[]:=
Out[]:=

? TNewTensor

Symbol

TNewTensor[**tensorID**, **metricID**, **coordinatesID**, **indices**, **components**, **symbol**] creates a new tensor object.

tensorID is a string that will be used to identify the new object, and must be unique.

metricID is the unique ID of a tensor object representing a metric, created using TNewMetric[]. The metric will be used to raise and lower indices for the new tensor.

coordinatesID is the unique ID of a tensor object representing a coordinate system, created using TNewCoordinates[]. This coordinate system will be used to specify the components of the new tensor. If omitted, the default coordinate system of the metric **metricID** will be used.

indices must be a list of the form {±1, ±1, ...}, where +1 corresponds to an upper index and -1 corresponds to a lower index.

components is a list specifying the representation of the tensor with the index configuration **indices** and in the coordinate system **coordinatesID**.

symbol will be used to represent the tensor in formulas. If not given, the placeholder □ will be used.

In OGRe, all tensor objects must have an **associated metric** - except coordinate objects, and the metric tensors themselves. This is because OGRe automatically raises and lowers indices as appropriate for various operations such as adding and contracting tensors, and it cannot do so without knowing which metric to use. Even scalars, which have no indices, should still be associated to a specific metric - since they can multiply other tensors, and you cannot multiply tensors from different manifolds together.

The index configuration of the tensor is a 1-dimensional **List**. The number of indices is the rank of the tensor. Each element in the **List** corresponds to one index, with +1 specifying an upper index and -1 specifying a lower index. For example, {-1, -1} corresponds to a tensor such as the metric $g_{\mu\nu}$, which has two lower indices, while {1, -1, -1, -1} corresponds to a tensor such as the Riemann tensor $R^\rho_{\sigma\mu\nu}$, which has one upper index followed by three lower indices.

The components of the tensor must also be a **List**. The components are the representation of the new tensor in the given index configuration and coordinate system. If a coordinate system is not specified, the default coordinate system of the associated metric will be used - but it is recommended to always specify the coordinate system explicitly, to avoid accidentally defining the tensor with the wrong components. The components will be automatically converted to different indices or coordinates later as needed, as we will demonstrate below.

To create a **scalar**, or a tensor of rank 0 (with no indices), we must input an empty list {} for the indices, and a list with one item for the components. For example, let us define the **Kretschmann**

scalar in the Schwarzschild spacetime (below we will show how to calculate it directly from the metric):

```
In[ ]:= TNewTensor["Kretschmann", "Schwarzschild", "Spherical", {}, { $\frac{48 M^2}{r^6}$ }, "K"]
```

```
Out[ ]:= Kretschmann
```

Again, the output is the unique ID of the tensor object that was created. Let us show the tensor:

```
In[ ]:= TShow["Kretschmann"]
```

OGRe: Kretschmann: $K(t, r, \theta, \phi) = \frac{48 M^2}{r^6}$

Notice that the output of **TNewTensor** is also the input of **TShow**, so in fact, we could **compose** them together using **@**. We will do so from now on.

Similarly, we can create a **vector**, or a tensor of rank 1 (with one index). For example, let us create a vector for the 4-velocity of a particle moving at 3-velocity v along the x direction in Minkowski space. (We do not need to reserve the symbol v , since we already reserved it for the Alcubierre metric above.)

Since the 4-velocity has an upper index by definition, we make sure to define the components in the representation of the tensor with an upper index by specifying the index configuration as **{1}**:

```
In[ ]:= TShow@TNewTensor["FourVelocity", "Minkowski", "Cartesian", {1}, { $\frac{1, v, 0, 0}{\sqrt{1-v^2}}$ }]
```

OGRe: FourVelocity: $\square^\mu(t, x, y, z) = \begin{pmatrix} \frac{1}{\sqrt{1-v^2}} \\ \frac{v}{\sqrt{1-v^2}} \\ 0 \\ 0 \end{pmatrix}$

Again, the output of **TNewTensor** was the ID of the tensor, **"FourVelocity"**, but that is also the input we want to pass to **TShow**, so we **composed** the two modules together. Note also that since we did not specify a symbol for this tensor, its symbol is just a placeholder $\square$. We will give it a proper symbol below.

Finally, as an example of a tensor of rank 2 (with two indices), let us define the stress-energy tensor $T^{\mu\nu}$ for a perfect fluid. First, let us reserve the symbols ρ (for the energy density) and p (for the pressure):

```
In[ ]:= TSetReservedSymbols[{ $\rho$ ,  $p$ }]
```

```
Out[ ]:= {t, x, y, z, r,  $\theta$ ,  $\phi$ , M, v, f,  $\rho$ , p}
```

Next we create the tensor, using its matrix representation with two upper indices by specifying the index configuration $\{1,1\}$:

In[]:=

```
TShow@TNewTensor["PerfectFluid", "Minkowski",
  "Cartesian", {1, 1}, DiagonalMatrix[{ρ, p, p, p}], "T"]
```

OGRe:

$$\text{PerfectFluid: } T^{\mu\nu}(t, x, y, z) = \begin{pmatrix} \rho & 0 & 0 & 0 \\ 0 & p & 0 & 0 \\ 0 & 0 & p & 0 \\ 0 & 0 & 0 & p \end{pmatrix}$$

In a similar manner, we could also define tensors of rank 3 and above. However, such tensors are most often derived by operating on lower-rank tensors, rather than defined manually via their components. We will see an example of such a derivation when we derive the Christoffel symbols and Riemann tensor from the metric below.

Tensor ID completion

Now is a great time to introduce a feature that makes OGRE very user-friendly and easy to use: **tensor ID completion**. All OGRE modules that take existing tensor IDs as arguments will show a dropdown menu with a list of relevant IDs - coordinate systems, metrics, and/or tensors - as soon as you type the opening double quotes for a string in a relevant argument position. Some other strings are also auto-completed, such as common symbols. For the modules we used so far, completions work as follows:

- For **TLineElement** and **TVolumeElement**, the first argument is the ID of an existing metric, so if you start typing **TLineElement["** or **TVolumeElement["** in a Mathematica input cell, as soon as you type the " to start a string, a list of all existing metric IDs will be shown for you to select from. Currently, this will show "Minkowski", "Schwarzschild", and "Alcubierre".
- For **TShow** and **TList**, the first argument is the ID of an existing tensor, so if you start typing **TShow["** or **TList["**, a list of all existing tensor IDs (including coordinate systems and metrics) will be shown. Currently, this will show the coordinate systems "Cartesian" and "Spherical", the metrics "Minkowski", "Schwarzschild", and "Alcubierre", and the tensors "Kretschmann", "FourVelocity", and "PerfectFluid".
- For **TNewMetric**, the first argument is the new ID you create, so it is not completed for you, but the second argument is the ID of an existing coordinate system. Therefore, if you start typing **TNewMetric["MyMetric", "**, a list of all existing coordinate system IDs will be shown. Currently, this will show "Cartesian" and "Spherical". Additionally, for the fourth argument, a list of common metric symbols will be shown.
- For **TNewTensor**, the first argument is the new ID you create, so it is not completed for you, but the second argument is the ID of an existing metric. Therefore, if you start typing **TNewTensor["MyTensor", "**, a list of all existing metric IDs will be shown. Similarly, the third argument is the ID of an existing coordinate system, so if you start typing **TNewTensor["MyTensor", "MyMetric", "**, a list of all existing coordinate system IDs will be shown.

Most of the modules we will introduce in the rest of this documentation will utilize tensor ID completion as well. It is highly recommended to try it for yourself now, so you can see how it works.

Operations on single tensors

Changing the symbol or ID of a tensor

If we ever want to change the symbol used to display a tensor, we can simply use the module **TChangeSymbol**:

In[]:= ? TChangeSymbol

Out[]:=

Symbol

TChangeSymbol[ID, symbol] changes the symbol of the tensor object ID to symbol.

▼

For example, let us give the symbol u to the four-velocity, and then show it:

In[]:= TShow@TChangeSymbol["FourVelocity", "u"]

OGRe:

$$\text{FourVelocity: } u^\mu(t, x, y, z) = \begin{pmatrix} \frac{1}{\sqrt{1-v^2}} \\ \frac{v}{\sqrt{1-v^2}} \\ 0 \\ 0 \end{pmatrix}$$

Similarly, we can also change the ID of the tensor using the **TChangeID** module:

In[]:= ? TChangeID

Out[]:=

Symbol

TChangeID[oldID, newID] changes the ID of the tensor object oldID to newID.

If the tensor is a metric or a coordinate system, all currently defined tensors will be scanned, and any references to oldID will be replaced with newID.

▼

For example, let us change the ID of the 4-velocity tensor from **"FourVelocity"** to **"4-Velocity"**:


```
In[ ]:= TChangeID["FourVelocity", "4-Velocity"]
Out[ ]:=
```

4-Velocity

The old ID no longer represents any tensor object, so we get an error if we try using it:

```
In[ ]:= TShow["FourVelocity"]
```

TMessage: The tensor "FourVelocity" does not exist.

```
Out[ ]:=
```

\$Aborted

Note that this error is associated with the symbol **TMessage**. Any message not associated with a specific OGRe module will be associated with this symbol.

We can access the tensor using the new ID:

```
In[ ]:= TShow["4-Velocity"]
```

OGRe:

$$4\text{-Velocity: } u^\mu(t, x, y, z) = \begin{pmatrix} \frac{1}{\sqrt{1-v^2}} \\ \frac{v}{\sqrt{1-v^2}} \\ 0 \\ 0 \end{pmatrix}$$

Note that when we define a tensor using a metric and a coordinate system, OGRe doesn't store the actual metric components or coordinates inside the tensor object - it only stores **references** to the relevant objects, using their IDs. This both improves performance and avoids redundancies that can lead to inconsistencies. For this reason, if the tensor to be renamed represents a metric or a coordinate system, OGRe will automatically update the references in the definitions of all of the tensors that have been defined so far in the session using that metric or coordinate system. This guarantees that there are never any broken references.

Deleting and overwriting tensors

If we want to delete a tensor we have created, we can use the **TDelete** module:

In[]:=
Out[]=

? TDelete

Symbol

TDelete[ID] permanently deletes the tensor object **ID**. If the tensor is a metric or coordinate system, it cannot be deleted unless all tensors referring to it have been deleted first.

To prevent breaking references, **TDelete** will not delete a tensor object representing a metric or coordinate system if it is referred to by any other tensor. For example, if we try to delete the coordinate system "**Cartesian**", we will get an error message, since it is used as the default coordinate system for some tensors:

In[]:=
Out[]=

TDelete["Cartesian"]

TDelete: The coordinate system "Cartesian" cannot be deleted, as it is the default coordinate system of the tensor "4-Velocity". To delete the coordinate system, first change the default coordinate system of "4-Velocity" and any other relevant tensors.

\$Aborted

Similarly, we cannot delete the metric "**Minkowski**" since it was used to define some tensors:

In[]:=
Out[]=

TDelete["Minkowski"]

TDelete: The metric "Minkowski" cannot be deleted, as it has been used to define the tensor "4-Velocity". To delete the metric, first delete "4-Velocity" and any other tensors defined using this metric.

\$Aborted

There is no module to change the components of a tensor after it has already been defined, as this may break class invariants (in other words, introduce inconsistencies in the data). Instead, you must create a new tensor with the same ID using **TNewTensor**. By default, OGRE does not allow overwriting tensors, to prevent loss of data. Users who want to be able to create new tensors with the same ID as an existing tensor without deleting the old tensor first, and are confident that they will not accidentally lose any data by doing so, may enable overwriting tensors using **TSetAllowOverwrite**:

In[]:=
Out[]:=

? TSetAllowOverwrite

Symbol

TSetAllowOverwrite[True] allows overwriting tensors. If the user creates a new tensor with the same ID as an existing tensor, the latter will be overwritten. Note that this can result in loss of data.

TSetAllowOverwrite[False] disallows overwriting, which is the default setting. TSetAllowOverwrite[] returns the current setting. Note that this setting is persistent between sessions.

First let us save the current state of TSetAllowOverwrite:

In[]:= currentOverwrite = TSetAllowOverwrite[];

Next, we turn it off:

In[]:= TSetAllowOverwrite[False]

TSetAllowOverwrite: Overwriting tensors turned off.

Overwriting the tensor is now forbidden:

In[]:= TNewTensor["4-Velocity", "Minkowski", "Cartesian", {1}, $\frac{\{1, v, 0, 0\}}{\sqrt{1 - v^2}}$, "v"]

... TMessage: A tensor with the ID "4-Velocity" already exists. Please rename it using TChangeID[] or delete it using TDelete[] first.
Type TSetAllowOverwrite[True] to allow overwriting tensors.

Out[]:=

\$Aborted

Let us now turn it on:

In[]:= TSetAllowOverwrite[True]

... TSetAllowOverwrite: Overwriting tensors turned on.

We can now overwrite the tensor "4-Velocity" by creating a new tensor with the same ID:

In[]:= TNewTensor["4-Velocity", "Minkowski", "Cartesian", {1}, $\frac{\{1, v, 0, 0\}}{\sqrt{1 - v^2}}$, "v"]

... TMessage: Overwriting the tensor "4-Velocity".

Out[]:=

4-Velocity

Like any other message, the message warning us that we are overwriting a tensor can be turned off using Off. In this case, the command to turn it off would be Off[TMessage::WarningOverwrite].

Note that the state of TSetAllowOverwrite is persistent between sessions - if you turn overwriting on, then it will remain on permanently, even in other Mathematica sessions, until you turn it back off. For

this reason, the documentation notebook will now restore the previous setting, to avoid messing up the preferences of users who evaluate it:

```
In[*]:= Off[TSetAllowOverwrite::Notify];
TSetAllowOverwrite[currentOverwrite];
On[TSetAllowOverwrite::Notify]
```

Raising and lowering indices

Raising and lowering indices is one of the most basic tensor operations. For example, if we have a vector represented with one upper index, v^ν , we can turn it into a covector, which is represented with one lower index, by **contracting** it with the metric:

$$v_\mu = g_{\mu\nu} v^\nu.$$

This is called "lowering an index". Here and in the rest of this documentation, we will be using the **Einstein summation convention**, where the same index repeated **exactly twice**, once as an upper index and once as a lower index, implies summation over that index. In this case, the implied summation is over $\nu \in \{0, 1, 2, 3\}$:

$$v_\mu = \sum_{\nu=0}^3 g_{\mu\nu} v^\nu = g_{\mu 0} v^0 + g_{\mu 1} v^1 + g_{\mu 2} v^2 + g_{\mu 3} v^3.$$

Such a sum over an index is called a contraction, and it is a generalization of the inner product, as we will describe in more detail below. Conversely, if we have a covector w_μ , we can raise its index by contracting it with the inverse metric:

$$w^\mu = g^{\mu\nu} w_\nu.$$

This works the same for indices of higher-rank tensors. For example, if we have a tensor of rank 2 represented with two upper indices, $T^{\mu\lambda}$, we can lower either one or both of its indices:

$$T^\mu_\nu = g_{\nu\lambda} T^{\mu\lambda}, \quad T_{\mu\nu} = g_{\mu\rho} g_{\nu\lambda} T^{\rho\lambda}.$$

In OGRE, since tensor objects are **abstract tensors**, independent of any specific index configuration, **there is no notion of raising or lowering the indices of a tensor object**. Instead, one simply requests to **display** the components of the tensor with the desired index configuration, without modifying the object itself. This works with both the **TShow** and **TList** modules, by simply adding as a second argument the list of indices, as when we created a new tensor.

As an example, let us show the vector **"4-Velocity"** with a lower index, that is, with index configuration **{-1}**:

In[]:=

TShow["4-Velocity", {-1}]

OGRE:

$$\text{4-Velocity: } v_{\mu}(t, x, y, z) = \begin{pmatrix} -\frac{1}{\sqrt{1-v^2}} \\ \frac{v}{\sqrt{1-v^2}} \\ 0 \\ 0 \end{pmatrix}$$

OGRE automatically knows to use the Minkowski metric to lower the index, which means that a minus sign has been added to the first component, as expected. Similarly, let us lower just the **second** index on **PerfectFluid**:

In[]:=

TList["PerfectFluid", {1, -1}]

OGRE:

$$\begin{aligned} \text{PerfectFluid:} \\ T^t_t &= -\rho \\ T^x_x = T^y_y = T^z_z &= p \end{aligned}$$

The components of the representation of the metric with two upper indices are the components of the inverse metric, since

$$g_{\mu\lambda} g^{\lambda\nu} = \delta_{\mu}^{\nu}.$$

Therefore, a quick way to show the components of the inverse metric is:

In[]:=

TShow["Schwarzschild", {1, 1}]

OGRE:

$$\text{Schwarzschild: } g^{\mu\nu}(t, r, \theta, \phi) = \begin{pmatrix} \frac{r}{2M-r} & 0 & 0 & 0 \\ 0 & 1 - \frac{2M}{r} & 0 & 0 \\ 0 & 0 & \frac{1}{r^2} & 0 \\ 0 & 0 & 0 & \frac{\text{Csc}[\theta]^2}{r^2} \end{pmatrix}$$

For the same reason, the metric with one upper and one lower index is just the identity matrix:

In[]:=

TList["Schwarzschild", {1, -1}]

OGRE:

$$\begin{aligned} \text{Schwarzschild:} \\ g^t_t = g^r_r = g^{\theta}_{\theta} = g^{\phi}_{\phi} &= 1 \end{aligned}$$

As explained above, if the modules **TShow** or **TList** are called with only the tensor ID, the tensor is displayed in its **default index configuration**, which is the one first used to define the tensor. So the 4-

velocity has one upper index by default, and the stress tensor has two upper indices by default, because that is how we initially defined them. However, the default indices can be changed using the module `TChangeDefaultIndices`:

```
In[ ]:= ? TChangeDefaultIndices
```

```
Out[ ]:=
```

Symbol

`TChangeDefaultIndices[ID, indices]` changes the default index configuration of the tensor object **ID** to **indices**.

indices must be a list of the form $\{\pm 1, \pm 1, \dots\}$, where +1 corresponds to an upper index and -1 corresponds to a lower index.

▼

For example, let us change the default indices of the perfect fluid stress tensor to two lower indices:

```
In[ ]:= TChangeDefaultIndices["PerfectFluid", {-1, -1}]
```

```
Out[ ]:=
```

PerfectFluid

Now, when we display the tensor using `TShow` with only the tensor ID, this is the index configuration that will be used:

```
In[ ]:= TShow["PerfectFluid"]
```

OGRe:

$$\text{PerfectFluid: } T_{\mu\nu}(t, x, y, z) = \begin{pmatrix} \rho & 0 & 0 & 0 \\ 0 & p & 0 & 0 \\ 0 & 0 & p & 0 \\ 0 & 0 & 0 & p \end{pmatrix}$$

Note that the default indices can only be changed for user-defined tensors; they cannot be changed for coordinates, metrics, or built-in tensors such as curvature tensors (which we will introduce later).

Coordinate transformations

The components of any tensor may be transformed from one coordinate system x^μ to another coordinate system $x^{\mu'}$ using the following prescription:

- For every lower index μ , add a factor of $\partial x^\mu / \partial x^{\mu'}$ (i.e. the derivative of the old coordinates with respect to the new, or the **Jacobian**).
- For every upper index μ , add a factor of $\partial x^{\mu'} / \partial x^\mu$ (i.e. the derivative of the new coordinates with respect to the old, or the inverse of the Jacobian).

For example, given a tensor with components $T_{\alpha\beta}$ in a coordinate system x^μ , we can transform to components $T_{\alpha'\beta'}$ in another coordinate system $x^{\mu'}$ as follows:

$$T_{\alpha' \beta'}(x^{\mu'}) = \frac{\partial x^{\alpha}}{\partial x^{\alpha'}} \frac{\partial x^{\beta}}{\partial x^{\beta'}} T_{\alpha \beta}(x^{\mu}).$$

For a general rank (p, q) tensor with p upper indices $\alpha_1, \dots, \alpha_p$ and q lower indices $\beta_1, \dots, \beta_q$, the transformation takes the form

$$T_{\beta'_1 \dots \beta'_q}^{\alpha'_1 \dots \alpha'_p}(x^{\mu'}) = \left(\frac{\partial x^{\alpha'_1}}{\partial x^{\alpha_1}} \dots \frac{\partial x^{\alpha'_p}}{\partial x^{\alpha_p}} \right) \left(\frac{\partial x^{\beta_1}}{\partial x^{\beta'_1}} \dots \frac{\partial x^{\beta_q}}{\partial x^{\beta'_q}} \right) T_{\beta_1 \dots \beta_q}^{\alpha_1 \dots \alpha_p}(x^{\mu}).$$

As a mnemonic for this formula, recall that two indices may only be contracted if one of them is an upper index and the other is a lower index. If an index is in the denominator of a derivative, then its role is reversed (upper $\leftrightarrow$ lower). Thus the old (non-primed) and new (primed) indices can only be in places that allow properly contracting the Jacobian or inverse Jacobian with the tensor. For example, α_1 is an upper index in T and therefore must be contracted with a lower index. Thus, ∂x^{α_1} must be in the denominator, to lower its index and allow it to be contracted with the tensor.

As we saw above, OGRE automatically knows how to raise or lower indices as needed using the appropriate metric. Similarly, any operation that requires transforming to another coordinate system will perform the transformation automatically behind the scenes. However, for this to happen, OGRE needs to know the appropriate **transformation rules**. These are defined between the tensor objects representing the coordinates, which were generated using the module **TNewCoordinates**. The rules for transforming from a source coordinate system to a target coordinate system are stored within the tensor object representing the source. This is done using the module **TAddCoordTransformation**:

In[]:=
Out[]=

? TAddCoordTransformation

Symbol

TAddCoordTransformation[sourceID, targetID, rules] adds a transformation from the coordinate system **sourceID** to the coordinate system **targetID**.

rules must be a list of transformation rules. For example, $\{x \rightarrow r \sin[\theta] \cos[\phi], y \rightarrow r \sin[\theta] \sin[\phi], z \rightarrow r \cos[\theta]\}$ is a transformation from Cartesian to spherical coordinates.

Let us add the rules to transform from Cartesian to spherical coordinates:

In[]:=

```
TAddCoordTransformation["Cartesian", "Spherical",
  {x -> r Sin[θ] Cos[φ], y -> r Sin[θ] Sin[φ], z -> r Cos[θ]}];
```

These will be stored in the data of the object **"Cartesian"**. Note that we did not have to input a rule for t , since in this case, it stays the same. Conversely, let us add the rules to transform from spherical to Cartesian coordinates:

```
In[ ]:= TAddCoordTransformation["Spherical", "Cartesian",
  {r →  $\sqrt{x^2 + y^2 + z^2}$ ,  $\theta \rightarrow \text{ArcCos}\left[\frac{z}{\sqrt{x^2 + y^2 + z^2}}\right]$ ,  $\phi \rightarrow \text{ArcTan}[x, y]$  }];
```

These will be stored in the data of the object **"Spherical"**. Now OGRE knows how to convert back and forth between these two coordinate systems - and this will happen automatically whenever required. We just needed to provide these rules once and for all, and any tensor initially defined in one coordinate system can now be automatically converted to the other.

As in the case of raising and lowering indices, displaying a tensor in a different coordinate system is a simple matter of calling the modules **TShow** or **TList** with an additional argument specifying the coordinate system to use. For example, let us show the Minkowski metric in spherical coordinates:

```
In[ ]:= TShow["Minkowski", "Spherical"]
```

OGRE:

$$\text{Minkowski: } \eta_{\mu\nu}(t, r, \theta, \phi) = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & r^2 & 0 \\ 0 & 0 & 0 & r^2 \sin[\theta]^2 \end{pmatrix}$$

We can also ask to see a tensor in a specific index configuration **and** a specific coordinate system; in this case, the indices must come before the coordinates in the argument list:

```
In[ ]:= TShow["PerfectFluid", {1, 1}, "Spherical"]
```

OGRE:

$$\text{PerfectFluid: } T^{\mu\nu}(t, r, \theta, \phi) = \begin{pmatrix} \rho & 0 & 0 & 0 \\ 0 & p & 0 & 0 \\ 0 & 0 & \frac{p}{r^2} & 0 \\ 0 & 0 & 0 & \frac{p \csc[\theta]^2}{r^2} \end{pmatrix}$$

The module **TList** works in exactly the same way, for example:

```
In[ ]:= TList["Kretschmann", "Cartesian"]
```

OGRE:

Kretschmann:

$$K = \frac{48 M^2}{(x^2 + y^2 + z^2)^3}$$

Just as with default indices, every tensor has a **default coordinate system**, which is, initially, the one we used to create it. We can change it using the module **TChangeDefaultCoords**, and then whenever we display the tensor, it will be displayed in that coordinate system if no other coordinate system is specified:

In[]:=
Out[]:=

? TChangeDefaultCoords

Symbol

TChangeDefaultCoords[**tensorID**, **coordinatesID**] changes the default coordinate system of the tensor object **tensorID** to **coordinatesID**.

For example, let's change the default coordinates of the perfect fluid stress tensor to spherical coordinates:

In[]:=
Out[]:=

TChangeDefaultCoords["PerfectFluid", "Spherical"]

PerfectFluid

Now, when we display the tensor using **TList** with only the tensor ID (or with just a choice of indices), this is the coordinate system that will be used:

In[]:=

TList["PerfectFluid"]

OGRe:

PerfectFluid:

$$T_{tt} = \rho$$

$$T_{rr} = p$$

$$T_{\theta\theta} = p r^2$$

$$T_{\phi\phi} = p r^2 \sin[\theta]^2$$

Applying replacement rules or a function to the tensor components

TList and **TShow** accept a rule or a list of rules as an argument. If provided, the replacement will be applied to each of the tensor's components, the components will be simplified, and the tensor will be displayed with the new components. Note that this only applies to displaying the components; the tensor data itself will not change.

This can be used to replace a particular symbol in the tensor with another symbol or numerical value. For example, perhaps we would like to display the value of the Kretschmann scalar for a particular choice of M and r :

In[]:=

TShow["Kretschmann", {M → 1, r → 10}]

OGRe:

$$\text{Kretschmann: } K(t, r, \theta, \phi) = \frac{3}{62500}$$

Or maybe we would like to display the perfect fluid stress tensor with p equal to ρ :

In[]:= **TList**["PerfectFluid", $p \rightarrow \rho$]

OGRe:

PerfectFluid:

$$T_{tt} = T_{rr} = \rho$$

$$T_{\theta\theta} = r^2 \rho$$

$$T_{\phi\phi} = r^2 \rho \sin[\theta]^2$$

The replacement rules can, of course, also be combined with a choice of indices and/or coordinates, as long as the rules come after the indices and/or coordinates in the argument list:

In[]:= **TList**["PerfectFluid", {1, 1}, "Cartesian", $p \rightarrow \rho$]

OGRe:

PerfectFluid:

$$T^{tt} = T^{xx} = T^{yy} = T^{zz} = \rho$$

Additionally, a function can be provided to **TList** and **TShow** as their last argument. If provided, the function will be applied to each of the tensor's components, the components will be simplified, and the tensor will be displayed with the new components. Providing a list of rules is equivalent to applying the function **ReplaceAll** with those rules.

As an example, let's square the components of the 4-velocity:

In[]:= **TShow**["4-Velocity", #1² &]

OGRe:

$$4\text{-Velocity: } v^\mu(t, x, y, z) = \begin{pmatrix} \frac{1}{1-v^2} \\ \frac{v^2}{1-v^2} \\ 0 \\ 0 \end{pmatrix}$$

Indices, coordinates, replacement rules, and functions can all be combined together. The full syntax is:

- **TList**[ID, indices, coordinatesID, rules, function]
- **TShow**[ID, indices, coordinatesID, rules, function]

The arguments must be provided in this exact order, but any of them can be omitted. So we can, for example, provide just **indices** and **rules**, or **coordinatesID**, **rules**, and **function**, and so on; any combination in the correct order will work. If both **rules** and **function** are given, the rules will be applied before the function.

Customizing the simplification function

Whenever OGRE performs an operation that creates or modifies tensor components, such as converting between index representations or coordinate systems, it automatically simplifies the result. The simplification function can be specified using the module **TSetSimplifyFunc**:

```
In[ ]:= ? TSetSimplifyFunc
Out[ ]=
```

Symbol

TSetSimplifyFunc[] shows the function used when simplifying expressions.

TSetSimplifyFunc[**function**] changes the simplification function. The function will be called as **function**[**expression**, **assumptions**], where **assumptions** are the user-defined simplification assumptions set using TSetAssumptions[].

TSetSimplifyFunc[None] disables simplification completely.

TSetSimplifyFunc[Automatic] resets the simplification function to the default: FullSimplify.

By default, OGRE uses **FullSimplify**. However, one possible use of **TSetSimplifyFunc** is to replace it with **Simplify**, which is a much faster simplification function that still does a good job in most cases. This might be necessary if simplification is taking too long or getting stuck.

You may also use your own custom-made simplification function, combining various Mathematica functions or your own code. In that case, your function should take two arguments: the expression to be simplified, and a list of assumptions. A simple example is using **FullSimplify** with specific options. For example, we can use **FullSimplify** with the option **TimeConstraint->5** to limit the time spent on simplification to 5 seconds:

```
In[ ]:= TSetSimplifyFunc[FullSimplify[#1, #2, TimeConstraint -> 5] &]
Out[ ]=
```

FullSimplify[#1, #2, TimeConstraint -> 5] &

In extreme situations, you may even want to cancel simplification altogether, which can be achieved using **TSetSimplifyFunc[None]**. To restore the simplification function to the default, **FullSimplify**, you can use **TSetSimplifyFunc[Automatic]**:

```
In[ ]:= TSetSimplifyFunc[Automatic]
Out[ ]=
```

FullSimplify

Note that changing the simplification function will **not** automatically apply it to any existing tensors. The reason is that when OGRE calculates the components of a tensor in a particular representation, it calculates them **once and for all**, and then saves them in the object's data to be reused later. This is done to improve performance, so that the components don't have to be recalculated every time they are needed. However, we can force re-simplification of the stored components of a specific tensor

using the module **TSimplify**:

```
In[ ]:= ? TSimplify
```

```
Out[ ]:=
```

Symbol

TSimplify[ID] simplifies all previously-calculated representations of the tensor object **ID** based on the user-defined simplification assumptions set using **TSetAssumptions[]**. To be used if the assumptions have changed after the components have already been calculated.

TSimplify[expression] simplifies **expression** based on the user-defined simplification assumptions. If **expression** is a list, the components will be simplified in parallel.

We will demonstrate how to use **TSimplify** in the next section.

Setting simplification assumptions

Often, coordinate transformations are only invertible for a specific range of coordinates. For example, let us define a new scalar in Minkowski space, which is equal to the spatial distance from the origin:

```
In[ ]:= TShow@TNewTensor["SpatialDistance", "Minkowski", "Cartesian", {}, {sqrt[x^2 + y^2 + z^2]}, "d"]
```

```
OGRe:
```

SpatialDistance: $d(t, x, y, z) = \sqrt{x^2 + y^2 + z^2}$

When we convert this scalar to spherical coordinates, we expect to get r , but instead we get the absolute value of r :

```
In[ ]:= TShow["SpatialDistance", "Spherical"]
```

```
OGRe:
```

SpatialDistance: $d(t, r, \theta, \phi) = \text{Abs}[r]$

As usual in Mathematica, such issues can be easily fixed by simplifying with the correct **assumptions**. By default, OGRE uses the simplification assumption that all variables (coordinates, parameters, etc.) are **real**. Therefore, Mathematica knows to simplify $\sqrt{r^2}$ to $|r|$. However, Mathematica doesn't automatically know that r is non-negative.

The user may specify which assumptions to pass to the simplification function using the module **TSetAssumptions**:

```
In[ ]:=
Out[ ]=
```

? TSetAssumptions

Symbol

TSetAssumptions[] shows the assumptions to be used when simplifying expressions.

TSetAssumptions[**assumptions**] appends new assumptions to the previously added assumptions.

TSetAssumptions[None] clears all previously added assumptions.

TSetAssumptions[!Reals] disables the default assumption that all variables are real, which secretly adds the assumption Element[_, Reals] to the list of assumptions. TSetAssumptions[Reals] re-enables this assumption.

The output of this module is always an Association indicating whether variables are assumed to be real and listing the user-defined assumptions.

Note that these assumptions will be globally applied to **all** tensor calculations, which is usually the desired behavior, since for example the assumption $r \geq 0$ should apply to all tensors that use spherical coordinates. Let us set this assumption now:

```
In[ ]:=
Out[ ]=
```

TSetAssumptions [r ≥ 0]

<| AssumeReal → True, User → {r ≥ 0} |>

In fact, it is good practice to set any assumptions regarding the coordinates **as soon as they are defined**, so we should have set this assumption already when we defined the spherical coordinates in the beginning of this documentation. From now on, this assumption will automatically be used by modules that perform any kind of calculations on tensors. However, if we now try to show the scalar again using TShow, we still get the same (non-simplified) result:

```
In[ ]:=
```

TShow["SpatialDistance", "Spherical"]

OGRe:

SpatialDistance: $d(t, r, \theta, \phi) = \text{Abs}[r]$

The reason is that, as explained above, OGRe does not automatically recalculate components it already calculated previously. In this case, since we already calculated the spatial distance in spherical coordinates when we showed it above - **before** we set the new simplification assumptions - that value has been saved, and will not be recalculated automatically, even though we now have new assumptions.

However, we can use TSimplify to force re-simplification and get the expected result:

```
In[ ]:=
```

TShow[TSimplify["SpatialDistance"], "Spherical"]

OGRe:

SpatialDistance: $d(t, r, \theta, \phi) = r$

Here, again, note that TSimplify returns the ID of the tensor it simplifies, so we can compose it with TShow if we want to show that same tensor. However, since we are now using TShow with a second

argument, we put `TSimplify["SpatialDistance"]` as the first argument, instead of composing it directly with `@`.

Finally, we note that if you are using non-real variables, you can disable the assumption that all variables are real using `TSetAssumptions[!Reals]`. If you later want to turn it back on, use `TSetAssumptions[Reals]`. The value of the key "AssumeReal" in the output of `TSetAssumptions` indicates whether this assumption is turned on.

Getting the components of a tensor

Sometimes you may want to extract the components of a tensor in a specific representation as a `List` so you can use them outside of this package, as regular Mathematica expressions rather than tensor objects. This is done using `TGetComponents`:

In[]:=
Out[]:=

? TGetComponents

Symbol

`TGetComponents[ID]` returns the components of the tensor object **ID** in its default index configuration and coordinate system.

`TGetComponents[ID, indices]` returns the components in the index configuration **indices**, which should be a list of the form $\{\pm 1, \pm 1, \dots\}$, where $+1$ corresponds to an upper index and -1 corresponds to a lower index.

`TGetComponents[ID, coordinatesID]` returns the components in the coordinate system **coordinatesID**.

`TGetComponents[ID, rules]` applies `ReplaceAll[rules]` to each of the tensor's elements, and then automatically simplifies them, before they are returned. **rules** can be either a single rule of the form **lhs** $\rightarrow$ **rhs** or a list of rules.

`TGetComponents[ID, function]` maps **function** to each of the tensor's elements, and then automatically simplifies them, before they are returned.

`TGetComponents[ID, indices, coordinatesID, rules, function]` does all of the above; any of the arguments can be omitted. If both **rules** and **function** are given, the rules will be applied before the function.

If **indices** and/or **coordinatesID** are omitted, the default values are used, and a message will let the user know which representation the components are given in, to avoid confusion.

For example:

In[]:=
Out[]:=

InverseSchwarzschild = TGetComponents["Schwarzschild", {1, 1}, "Spherical"]

$$\left\{ \left\{ \frac{r}{2M-r}, 0, 0, 0 \right\}, \left\{ 0, 1 - \frac{2M}{r}, 0, 0 \right\}, \left\{ 0, 0, \frac{1}{r^2}, 0 \right\}, \left\{ 0, 0, 0, \frac{\csc[\theta]^2}{r^2} \right\} \right\}$$

We can now treat `InverseSchwarzschild` as any other `List` in Mathematica - for example, extract the element at a particular position:

In[]:=

`InverseSchwarzschild[[1, 1]]`

Out[]:=

$$\frac{r}{2M - r}$$

If the desired index configuration and/or coordinate system are not specified, the default ones will be used. However, it is important to always know exactly which representation the components are in, to avoid confusion. Thus, you will be notified which representation was used:

In[]:=

`TGetComponents["Schwarzschild"]`

TGetComponents: Using the default index configuration $\{-1, -1\}$ and the default coordinate system "Spherical".

Out[]:=

$$\left\{ \left\{ -1 + \frac{2M}{r}, 0, 0, 0 \right\}, \left\{ 0, \frac{1}{1 - \frac{2M}{r}}, 0, 0 \right\}, \left\{ 0, 0, r^2, 0 \right\}, \left\{ 0, 0, 0, r^2 \sin^2[\theta] \right\} \right\}$$

`TGetComponents` can take replacement rules and/or a function to map to the components, in the exact same syntax as `TList` and `TShow` (see above). For example, here are the components of the Schwarzschild metric on the hypersurface with $\theta = \pi/2$:

In[]:=

`TGetComponents["Schwarzschild",  $\theta \rightarrow \frac{\pi}{2}$ ]`

TGetComponents: Using the default index configuration $\{-1, -1\}$ and the default coordinate system "Spherical".

Out[]:=

$$\left\{ \left\{ -1 + \frac{2M}{r}, 0, 0, 0 \right\}, \left\{ 0, \frac{1}{1 - \frac{2M}{r}}, 0, 0 \right\}, \left\{ 0, 0, r^2, 0 \right\}, \left\{ 0, 0, 0, r^2 \right\} \right\}$$

Getting the dimension and rank of a tensor

The `TDim` module can be used to get the dimension, or number of coordinates, of the manifold a tensor is defined in.

In[]:=

`? TDim`

Out[]:=

Symbol

`TDim[ID]` returns the number of dimensions in the manifold that the tensor object `ID` is defined in, that is, the number of coordinates.

▼

For example:

```
In[ ]:= TDim["Minkowski"]
Out[ ]=
```

4

The **TRank** module can be used to get the rank of a tensor. The rank is the number of indices in the representations of the tensor. A rank-0 tensor is a scalar, a rank-1 tensor is a vector, a rank-2 tensor is a matrix, and so on.

```
In[ ]:= ? TRank
Out[ ]=
```

Symbol

TRank[ID] returns the rank of the tensor object **ID**, that is, the number of indices.

▼

```
In[ ]:= TRank["PerfectFluid"]
Out[ ]=
```

2

Getting information about tensors

The **TInfo** module can be used to display information about any tensor object, including its symbol, role, rank, dimensions, associated metric, and default coordinates and indices:

```
In[ ]:= ? TInfo
Out[ ]=
```

Symbol

TInfo[] lists all the tensors created so far in this session: coordinate systems, metrics, and the tensors associated with each metric.

TInfo[ID] displays information about the tensor object **ID**, including its symbol, role, rank, dimensions, associated metric, and default coordinates and indices, in human-readable form.

If **ID** represents a coordinate system, displays a list of all tensors using it as their default coordinate system.

If **ID** represents a metric, displays a list of all tensors using it as their associated metric.

▼

TInfo will also tell you which other tensor objects use this tensor as their metric or default coordinate system, if applicable. Here are some examples:

In[]:=

TInfo["Cartesian"]

ID: Cartesian
Symbol: x
Role: Coordinates
Rank: 1
Dimensions: 4
Default Indices: {1}
Default Coordinates For: 4-Velocity, Alcubierre, Minkowski, SpatialDistance
Components: [TList](#) | [TShow](#)

In[]:=

TInfo["Minkowski"]

ID: Minkowski
Symbol: η
Role: Metric
Rank: 2
Dimensions: 4
Default Coordinates: Cartesian
Default Indices: {-1, -1}
Tensors Using This Metric: 4-Velocity | [PerfectFluid](#) | [SpatialDistance](#)
Components: [TList](#) | [TShow](#)

In[]:=

TInfo["PerfectFluid"]

ID: PerfectFluid
Symbol: T
Role: Tensor
Rank: 2
Dimensions: 4
Metric: Minkowski
Default Coordinates: Spherical
Default Indices: {-1, -1}
Components: [TList](#) | [TShow](#)

You can click on the links to [TList](#) and [TShow](#) at the bottom to display the components of the tensor being inspected. You can also click on any tensor name in the output (e.g. [Minkowski](#), or [4-Velocity](#) in **TInfo["Cartesian"]**) to execute **TInfo** for that tensor.

In addition, calling **TInfo[]** with no parameters lists all the tensors created so far in this session - coordinate systems, metrics, and the tensors associated with each metric:

In[]:=

TInfo[]

Total tensors created: 9

Coordinate Systems:

1. Cartesian
2. Spherical

OGRe:

Metrics:

1. **Alcubierre** →
2. **Minkowski** → 4-Velocity | PerfectFluid | SpatialDistance
3. **Schwarzschild** → Kretschmann

We see that we created 9 tensors in total so far: 2 coordinate systems, 3 metrics, 3 tensors associated with the Minkowski metric, and 1 tensor associated with the Schwarzschild metric.

Calculations with tensors

The TCalc module

Now that we have all the bookkeeping of tensors out of the way, we can finally discuss how to use those tensors in calculations. In OGRE, all tensor calculations are performed using the **TCalc** module:

In[]:=

? TCalc

Out[]=

Symbol

TCalc[**formula**] calculates a tensor **formula**, which may involve any number of tensors in the format

ID[**indices**], where **ID** is a tensor object and **indices** is a string representing the order of indices, along with any combination of the following operations:

- Addition: For example, "A"[" $\mu\nu$ "] + "B"[" $\mu\nu$ "].
- Contraction: For example, "A"[" $\mu\lambda$ "] . "B"[" $\lambda\nu$ "].
- Multiplication by scalar: For example, 2 * "A"[" $\mu\nu$ "].

TCalc[**targetID**[**targetIndices**], **formula**, **symbol**] calculates a tensor **formula** and stores the result in a new tensor object.

targetID specifies the ID of the tensor object in which to store the result. If omitted, the ID "Result" will be used.

targetIndices specifies the order of indices of the resulting tensor. The indices must be a permutation of the free indices of **formula**. If omitted, the indices are assumed to be in the same order as they appear in **formula**.

symbol specifies the symbol to use for the resulting tensor. If omitted, the placeholder symbol $\square$ will be used.

Any use of `TCalc` should be thought of as invoking a tensor equation of the form

$$L_{\beta_1 \dots \beta_q}^{\alpha_1 \dots \alpha_p} = R_{\beta_1 \dots \beta_q}^{\alpha_1 \dots \alpha_p},$$

where both the left-hand side and the right-hand side are tensors of the same rank and with the same **free indices** (that is, indices that are not being contracted). $L_{\beta_1 \dots \beta_q}^{\alpha_1 \dots \alpha_p}$ is the tensor that will be used to store the result, while $R_{\beta_1 \dots \beta_q}^{\alpha_1 \dots \alpha_p}$ is (the final result of) a general tensor calculation which contains any combination of addition, multiplication by a scalar, trace, contraction, partial derivative, and covariant derivative. Let us now go over these operations one by one, and give some examples.

Addition of tensors

Addition of tensors in OGRE is represented by a sum of the form `"ID1"["indices1"] + "ID2"["indices2"]`, where `"ID1"` and `"ID2"` are the IDs of the tensor objects to be added, and `"indices1"` and `"indices2"` are the **index specifications** for each tensor, given as a string of letters. Note that you do **not** specify the position (upper or lower) of the indices. Furthermore, just like in any tensor equation, **the index letters themselves have no meaning**; they are just placeholders. Therefore, `"αβγ"`, `"abc"`, and `"ABC"` are all completely equivalent. The only requirement is that the **indices are consistent**; in the case of addition, this means that both tensors being added must have **the same indices up to permutation**.

The following constraints apply to addition of tensors:

- You may not add a tensor representing a coordinate system to any other tensor, since coordinates do not transform like tensors.
- You may not add two tensors associated with different metrics, since their sum would have undefined transformation properties.
- You may not add two tensors with different ranks, since that is not a well-defined operation.
- As stated above, both tensors must have the same indices up to permutation. $A^{\mu\nu} + B^{\mu\nu}$ and $A^{\mu\nu} + B^{\nu\mu}$ are both okay, but $A^{\mu\nu} + B^{\alpha\beta}$ doesn't make sense, as it has more free indices than the rank of the result.

As an example, let us add the Minkowski metric $\eta_{\mu\nu}$ and the perfect fluid stress tensor $T_{\mu\nu}$:

<code>In[]:=</code>	<code>TShow@TCalc["Minkowski"["μν"] + "PerfectFluid"["μν"]]</code>
<code>OGRe:</code>	Result: $\square_{\mu\nu}(t, x, y, z) = \begin{pmatrix} -1 + \rho & 0 & 0 & 0 \\ 0 & 1 + p & 0 & 0 \\ 0 & 0 & 1 + p & 0 \\ 0 & 0 & 0 & 1 + p \end{pmatrix}$

Notice that the result was stored in a tensor with ID `"Result"`, and has no symbol. We can add a symbol to use as an additional argument:

```
In[ ]:= TShow@TCalc["Minkowski"["μν"] + "PerfectFluid"["μν"], "S"]
```

OGRe:

$$\text{Result: } S_{\mu\nu}(t, x, y, z) = \begin{pmatrix} -1+\rho & 0 & 0 & 0 \\ 0 & 1+p & 0 & 0 \\ 0 & 0 & 1+p & 0 \\ 0 & 0 & 0 & 1+p \end{pmatrix}$$

With this symbol, the tensor equation we are calculating becomes:

$$S_{\mu\nu} = \eta_{\mu\nu} + T_{\mu\nu}.$$

We can also use a different ID for the result by giving it as the first argument, with or without a symbol:

```
In[ ]:= TShow@TCalc["SumResult", "Minkowski"["μν"] + "PerfectFluid"["μν"], "S"]
TDelete["SumResult"]
```

OGRe:

$$\text{SumResult: } S_{\mu\nu}(t, x, y, z) = \begin{pmatrix} -1+\rho & 0 & 0 & 0 \\ 0 & 1+p & 0 & 0 \\ 0 & 0 & 1+p & 0 \\ 0 & 0 & 0 & 1+p \end{pmatrix}$$

(We deleted the result to avoid cluttering memory with unused objects.) In the following examples, we will not specify a symbol, to keep the code cleaner.

Sometimes it is also helpful to specify indices for the result. To give an example, let us define the following non-symmetric tensor:

```
In[ ]:= TShow@TNewTensor["NonSymmetric", "Minkowski", "Cartesian",
{-1, -1}, {{0, 0, 0, 1}, {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}}, "N"]
```

OGRe:

$$\text{NonSymmetric: } N_{\mu\nu}(t, x, y, z) = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

If we add it to the Minkowski metric, we get:

$$\square_{\mu\nu} = \eta_{\mu\nu} + N_{\mu\nu},$$

In[]:=

TShow@TCalc["Minkowski"["μν"] + "NonSymmetric"["μν"]]

OGRE:

$$\text{Result: } \square_{\mu\nu}(t, x, y, z) = \begin{pmatrix} -1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

However, if we flip its index string from "μν" to "νμ", then we instead get:

$$\square_{\mu\nu} = \eta_{\mu\nu} + N_{\nu\mu},$$

In[]:=

TShow@TCalc["Minkowski"["μν"] + "NonSymmetric"["νμ"]]

OGRE:

$$\text{Result: } \square_{\mu\nu}(t, x, y, z) = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}$$

Since the order of indices now matters, we can also define an index string for the left-hand side, to indicate the order of indices we want in the result. If that string is also "νμ", then we get back to the original result:

$$\square_{\nu\mu} = \eta_{\mu\nu} + N_{\nu\mu},$$

In[]:=

TShow@TCalc["Result"["νμ"], "Minkowski"["μν"] + "NonSymmetric"["νμ"]]

OGRE:

$$\text{Result: } \square_{\mu\nu}(t, x, y, z) = \begin{pmatrix} -1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

We see that explicitly specifying the indices in TCalc allows it to have a **1-to-1 correspondence with any tensor equation**. Importantly, note that there is no difference between "NonSymmetric"["μν"] and "NonSymmetric"["νμ"] on its own, as **the index labels themselves are meaningless** unless there is some context in which they obtain meaning - as is always the case for tensor expressions. However, there is a big difference between, for example, "Minkowski"["μν"] + "NonSymmetric"["μν"] and "Minkowski"["μν"] + "NonSymmetric"["νμ"], as the indices have a different order, and thus the two expressions refer to adding different components.

Of course, any number of tensors can be added, not just two - and the same tensor can be added multiple times, with different index specifications each time. For example, we can calculate:

$$\square_{\mu\nu} = \eta_{\mu\nu} + T_{\mu\nu} + N_{\mu\nu} + N_{\nu\mu},$$

In[]:=

```
TShow@TCalc["Minkowski"["μν"] +
  "PerfectFluid"["μν"] + "NonSymmetric"["μν"] + "NonSymmetric"["νμ"]]
```

OGRe:

$$\text{Result: } \square_{\mu\nu}(t, x, y, z) = \begin{pmatrix} -1+\rho & 0 & 0 & 1 \\ 0 & 1+p & 0 & 0 \\ 0 & 0 & 1+p & 0 \\ 1 & 0 & 0 & 1+p \end{pmatrix}$$

Multiplication of a tensor by a scalar

Multiplication of a tensor by a scalar in OGRE is represented by a product of the form **scalar** * **"ID"["indices"]**, where **"ID"** is the ID of the tensor object to be multiplied, **"indices"** is an index specification as for addition, and **scalar** is the scalar to multiply by. Note that **scalar** should be a normal Mathematica symbol, such as a number or a variable, and **not** a tensor object of rank 0. To multiply a tensor by a tensor of rank 0, use contraction instead, as detailed in the next section.

As an example, let us multiply the Minkowski metric $\eta_{\mu\nu}$ by 2. The tensor equation we will be calculating is:

$$\square_{\mu\nu} = 2 \eta_{\mu\nu},$$

and the OGRE expression to calculate it (and show the result) is:

In[]:=

```
TShow@TCalc[2 "Minkowski"["μν"]]
```

OGRe:

$$\text{Result: } \square_{\mu\nu}(t, x, y, z) = \begin{pmatrix} -2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}$$

While in this example the indices seem redundant, they are necessary because in most non-trivial situations we would like to combine multiplication with other operations, such as addition or contraction, in which the order of indices matters. For example, consider:

$$\square_{\mu\nu} = 2 t \eta_{\mu\nu} - 3 x T_{\mu\nu} + 4 y N_{\mu\nu} - 5 z N_{\nu\mu},$$

In[]:=

```
TShow[TCalc[2 t "Minkowski"["μν"] - 3 x "PerfectFluid"["μν"] +
4 y "NonSymmetric"["μν"] - 5 z "NonSymmetric"["νμ"]]]
```

OGRE:

$$\text{Result: } \square_{\mu\nu}(t, x, y, z) = \begin{pmatrix} -2t - 3x\rho & 0 & 0 & 4y \\ 0 & 2t - 3px & 0 & 0 \\ 0 & 0 & 2t - 3px & 0 \\ -5z & 0 & 0 & 2t - 3px \end{pmatrix}$$

Taking traces and contracting tensors

The most complicated tensor operation is **contraction**, a generalization of the vector inner product. This is done by summing over one or more disjoint pairs of indices, with each pair containing exactly one upper index and one lower index. Raising and lowering indices is one example of contraction: the metric (or its inverse) is contracted with a tensor. Coordinate transformations are another example, where we contract the Jacobian (or its inverse) with a tensor.

The simplest example of contraction is the **vector inner product**, which is defined as the contraction of a vector (one upper index) with a covector (one lower index):

$$v^\mu w_\mu = g_{\mu\nu} v^\mu w^\nu = g(v, w).$$

The middle part of this equality comes from the fact that, as explained above, when we lower an index on w^ν , we use the metric:

$$w_\mu = g_{\mu\nu} w^\nu.$$

This, in turn, justifies the notation $g(v, w)$ on the right-hand side, as this is, in fact, an inner product of two vectors using the metric g (in index-free notation).

Contraction of indices in higher-rank tensors is simply a generalization of the inner product, for example:

$$A^{\mu\alpha} B_{\alpha\nu} = g_{\alpha\beta} A^{\mu\alpha} B^\beta{}_\nu.$$

We can also contract more than one index:

$$A^{\mu\nu} B_{\mu\nu} = g_{\mu\alpha} g_{\nu\beta} A^{\mu\nu} B^{\alpha\beta}.$$

This simply amounts to the fact that lowering both indices of $B^{\alpha\beta}$ involves contracting each index with the metric. We can even contract two indices **of the same tensor**:

$$A^\mu{}_\mu = g_{\mu\nu} A^{\mu\nu}.$$

This is called **taking the trace**. Furthermore, it is also possible to contract pairs of indices from more than two tensors at the same time:

$$A^{\mu\nu} B_{\nu\rho} C^{\rho\sigma} = g_{\nu\alpha} g_{\rho\beta} A^{\mu\nu} B^{\alpha\beta} C^{\rho\sigma}.$$

However, such operations can always be broken down into individual contractions of pairs of tensors.

For example, in this case, one could first contract $B_{\nu\rho}$ with $C^{\rho\sigma}$ and then contract the result with $A^{\mu\nu}$ - which is indeed how this kind of contraction will be performed in OGRE in practice:

$$A^{\mu\nu} B_{\nu\rho} C^{\rho\sigma} = A^{\mu\nu} (B_{\nu\rho} C^{\rho\sigma}).$$

In a contraction, there are two types of indices: **contracted indices**, which are summed upon, and **free indices**, which are not summed upon. The rank of the tensor that results from the contraction is the number of free indices. So for example, in the expression $A^{\mu\alpha} B_{\alpha\nu}$ we have one contracted index, α , and two free indices, μ and ν . Therefore, the resulting tensor is of rank two: $T^\mu_\nu = A^{\mu\alpha} B_{\alpha\nu}$.

Contraction of tensors in OGRE is represented by an expression of the form `"ID1"["indices1"] . "ID2"["indices2"]`, where `"ID1"` and `"ID2"` are the IDs of the tensor objects to be contracted, and `"indices1"` and `"indices2"` are the index strings for each tensor. Any **matching indices** in both index strings will be contracted. This means that, for example, $v^\mu w_\mu$ is calculated using `"v"["μ"] . "w"["μ"]` and $A^{\mu\nu} B_{\nu\rho} C^{\rho\sigma}$ is calculated using `"A"["μν"] . "B"["νρ"] . "C"["ρσ"]`. Note that the user doesn't need to worry about the contracted indices being one upper and one lower, which is a common source of errors when contracting tensors by hand; the order of the indices, and whether the same index repeats twice, is all that matters.

As a first example, let us create the stress-energy tensor for a perfect fluid with a 4-velocity u^μ . This is defined as follows:

$$T^{\mu\nu} = (\rho + p) u^\mu u^\nu + p g^{\mu\nu}.$$

Even though this does not involve any contractions, it still counts as a "trivial" contraction, since two tensors (the 4-velocities) are juxtaposed next to each other to create another tensor. This is also known as an **outer product**. Therefore, it uses the same dot product syntax as any other contraction, except that there are **no matching indices**. Note that this expression involves not just contraction (in the first term), but also multiplication by a scalar (in both terms), and addition of the two terms together. Again, OGRE takes care of everything behind the scenes, so this just works:

In[]:=

```
TShow@TCalc["PerfectFluidFromVelocity",
  (\rho + p) "4-Velocity"["μ"] . "4-Velocity"["ν"] + p "Minkowski"["μν"], "T"]
```

OGRE:

$$\text{PerfectFluidFromVelocity: } T_{\mu\nu}(t, x, y, z) = \begin{pmatrix} \frac{p v^2 + \rho}{1 - v^2} & \frac{v(p + \rho)}{-1 + v^2} & 0 & 0 \\ \frac{v(p + \rho)}{-1 + v^2} & \frac{p + v^2 \rho}{1 - v^2} & 0 & 0 \\ 0 & 0 & p & 0 \\ 0 & 0 & 0 & p \end{pmatrix}$$

Indeed, for $v = 0$ we get the previously defined stress tensor:

In[]:= TShow["PerfectFluidFromVelocity", $v \rightarrow 0$]

OGRe:

$$\text{PerfectFluidFromVelocity: } T_{\mu\nu}(t, x, y, z) = \begin{pmatrix} \rho & 0 & 0 & 0 \\ 0 & p & 0 & 0 \\ 0 & 0 & p & 0 \\ 0 & 0 & 0 & p \end{pmatrix}$$

Note that here, the second argument is a replacement rule to apply to the tensor elements, as explained above under "applying replacement rules or a function to the tensor components".

Multiplying a tensor by a scalar tensor (i.e. a tensor of rank 0) is also done using a "trivial" contraction with no contracted indices. For example:

In[]:= TShow[TCalc["SpatialDistance"[""]."Minkowski"[" $\mu\nu$ "]], "Spherical"]

OGRe:

$$\text{Result: } \square_{\mu\nu}(t, r, \theta, \phi) = \begin{pmatrix} -r & 0 & 0 & 0 \\ 0 & r & 0 & 0 \\ 0 & 0 & r^3 & 0 \\ 0 & 0 & 0 & r^3 \sin^2[\theta] \end{pmatrix}$$

Note the empty index string [""], which is mandatory in order for OGRe to recognize that the scalar is a tensor object. We can also multiply a scalar by another scalar:

In[]:= TShow[TCalc["SpatialDistance"[""]."SpatialDistance"[""]], "Spherical"]

OGRe:

$$\text{Result: } \square(t, r, \theta, \phi) = r^2$$

Now let us demonstrate some non-trivial contractions. First, we have the inner product of vectors - in this case, we get the norm (squared) of the 4-velocity, since we are contracting it with itself:

In[]:= TShow@TCalc["4-Velocity"[" μ "]."4-Velocity"[" μ "]]

OGRe:

$$\text{Result: } \square(t, x, y, z) = -1$$

Note that this can be done more easily with **TNormSquared** (see below). We can also contract several tensors together, with **two** matching pairs of indices:

```
In[ ]:= TShow@TCalc["4-Velocity"["μ"]."PerfectFluidFromVelocity"["μν"]."NonSymmetric"["νρ"]]
```

OGRe:

$$\text{Result: } \square_{\mu}(t, x, y, z) = \begin{pmatrix} 0 \\ 0 \\ 0 \\ -\frac{\rho}{\sqrt{1-v^2}} \end{pmatrix}$$

Finally, to take the trace of a tensor, we simply match pairs of indices in that tensor's index string:

```
In[ ]:= TShow@TCalc["Minkowski"["μμ"]]
```

OGRe:

$$\text{Result: } \square(t, x, y, z) = 4$$

```
In[ ]:= TShow@TCalc["PerfectFluidFromVelocity"["μμ"]]
```

OGRe:

$$\text{Result: } \square(t, x, y, z) = 3p - \rho$$

Of course, this also works for tensors with more than two indices, as we will see below. Any combination of indices can be used, with no limit on the number of traces taken for each tensor.

Calculating the norm squared

The norm squared of a tensor of any rank can be calculated with **TNormSquared**:

```
In[ ]:= ? TNormSquared
```

Out[]:=

Symbol

TNormSquared[*tensorID*] calculates the norm-squared of the tensor **tensorID** with respect to its metric, that is, the tensor contracted with itself in all indices. For example, for a vector v^a the norm-squared will be $v^a v_a$ and for a rank-2 tensor T^{ab} the result will be $T^{ab} T_{ab}$.

TNormSquared[*tensorID*, *coordinatesID*] calculates the norm-squared in the coordinate system **coordinatesID**.

For any tensor, the norm squared is defined as the tensor contracted with itself in all indices:

$$|v|^2 \equiv v^\mu v_\mu,$$

$$|T|^2 \equiv T^{\mu\nu} T_{\mu\nu},$$

and so on. The result is always a scalar expression. For example, the norm of any 4-velocity is -1:

```
In[ ]:= TNormSquared["4-Velocity"]
```

```
Out[ ]:=
```

-1

The norm of any metric is the number of dimensions in that metric:

```
In[ ]:= TNormSquared["Minkowski"]
```

```
Out[ ]:=
```

4

Derivatives and curvature tensors

The **partial derivative** ∂_μ is represented in OGRE using the symbol **TPartialD**. It can be contracted with other tensors using the usual OGRE contraction notation - including an appropriate index string - to calculate gradients and divergences.

```
In[ ]:= ? TPartialD
```

```
Out[ ]:=
```

Symbol

TPartialD[index] represents the partial derivative when used in a tensor expression given to TCalc[].

▼

The **gradient** of a tensor is the partial derivative ∂_μ acting on the tensor with a free index, e.g. $\partial_\mu \phi$, $\partial_\mu v^\nu$, or $\partial_\mu T^{\nu\lambda}$, resulting in a tensor of **one rank higher**. For example, we can calculate the gradient $\partial_\mu K$ of the Kretschmann scalar as follows:

```
In[ ]:= TShow@TCalc[TPartialD["μ"]."Kretschmann"[""]]
```

OGRE:

$$\text{Result: } \partial_\mu(t, r, \theta, \phi) = \begin{pmatrix} 0 \\ -\frac{288 M^2}{r^7} \\ 0 \\ 0 \end{pmatrix}$$

The **divergence** of a tensor is the contraction of the partial derivative ∂_μ with one of the tensor's indices, e.g. $\partial_\mu v^\mu$ or $\partial_\mu T^{\mu\nu}$, resulting in a tensor of **one rank lower**. For example, here is the divergence $\partial_\mu x^\mu$ of the Cartesian position vector:

In[]:=

TShow@TNewTensor["Position", "Minkowski", "Cartesian", {1}, {t, x, y, z}, "x"]

OGRe:

$$\text{Position: } x^\mu(t, x, y, z) = \begin{pmatrix} t \\ x \\ y \\ z \end{pmatrix}$$

In[]:=

TShow@TCalc[TPartialD["μ"]."Position"["μ"]]

OGRe:

Result: $\square(t, x, y, z) = 4$

As you can see, the syntax for both the gradient and divergence is the same; if the index specification of `TPartialD["μ"]` matches one of the indices of the tensor to its right, then the divergence will be calculated, otherwise the gradient will be calculated.

WARNING: When applying partial derivatives to tensors, the result generally does not transform like a tensor under a coordinate transformation. For this reason, in general relativity we normally use the **covariant derivative** (see below) instead of a partial derivative. However, there are three important exceptions, where partial derivatives must be used: in the covariant derivative itself, the **Levi-Civita connection**, and the **Riemann tensor**, all of which will be discussed below.

Of these three special cases, the covariant derivative and the Riemann tensor turn out to nonetheless transform like tensors under coordinate transformations, due to cancellations. However, the Levi-Civita connection, whose components are called the **Christoffel symbols**, does not transform like a tensor; it has a special transformation rule with an extra term, as we will see below.

In all other cases, if the user creates an arbitrary tensor using partial derivatives, the result will generally **transform incorrectly** under a coordinate transformation in OGRE. Therefore, it is highly recommended to avoid using partial derivatives with `TCalc` unless you really know what you're doing.

The Christoffel symbols

The **Christoffel symbols** are a very important tensor-like object in differential geometry. They are the components of the **Levi-Civita connection**, which is the unique torsion-free connection that preserves the metric. The Christoffel symbols are defined as follows:

$$\Gamma_{\mu\nu}^\lambda = \frac{1}{2} g^{\lambda\sigma} (\partial_\mu g_{\nu\sigma} + \partial_\nu g_{\sigma\mu} - \partial_\sigma g_{\mu\nu}).$$

Each of the terms inside the parentheses is a gradient of the metric, with different indices. For example, the first term $\partial_\mu g_{\nu\sigma}$ is represented in OGRE as `TPartialD["μ"]."Metric"["νσ"]` where `"Metric"` is the tensor object representing the metric. Since contraction, addition, and multiplication by a scalar can be combined arbitrarily when using `TCalc`, we can calculate the Christoffel symbols in a straightforward way as follows:

In[]:=

```

TList@TChangeDefaultIndices[TCalc[ $\frac{1}{2}$  "Schwarzschild"[" $\lambda\sigma$ "].
(TPartialD[" $\mu$ "]."Schwarzschild"[" $\nu\sigma$ "] + TPartialD[" $\nu$ "]."Schwarzschild"[" $\sigma\mu$ "] -
TPartialD[" $\sigma$ "]."Schwarzschild"[" $\mu\nu$ "]), "r"], {1, -1, -1}]
    
```

Result:

$$\begin{aligned}
 \Gamma_{tr}^t &= \Gamma_{rt}^t = -\Gamma_{rr}^r &= \frac{M}{r(-2M+r)} \\
 \Gamma_{tt}^r &= \frac{M(-2M+r)}{r^3} \\
 \Gamma_{\theta\theta}^r &= 2M - r \\
 \Gamma_{\phi\phi}^r &= (2M - r) \sin^2[\theta] \\
 \Gamma_{r\theta}^\theta &= \Gamma_{\theta r}^\theta = \Gamma_{r\phi}^\phi = \Gamma_{\phi r}^\phi = \frac{1}{r} \\
 \Gamma_{\phi\phi}^\theta &= -\cos[\theta] \sin[\theta] \\
 \Gamma_{\theta\phi}^\phi &= \Gamma_{\phi\theta}^\phi = \cot[\theta]
 \end{aligned}$$

OGRE:

Note that here we used `TChangeDefaultIndices` to ensure the result has the correct index placement (one upper, two lower).

However, there is a problem; as we mentioned above, **the Christoffel symbols are not the components of a tensor**, meaning that the Levi-Civita connection does not transform as a tensor does under a coordinate transformation. Indeed, by transforming the metric in the definition, one can show that

$$\Gamma_{\mu'\nu'}^{\lambda'} = \frac{\partial x^{\lambda'}}{\partial x^\lambda} \frac{\partial x^\mu}{\partial x^{\mu'}} \frac{\partial x^\nu}{\partial x^{\nu'}} \Gamma_{\mu\nu}^\lambda + \frac{\partial x^{\lambda'}}{\partial x^\lambda} \frac{\partial^2 x^\lambda}{\partial x^{\mu'} \partial x^{\nu'}}.$$

The first term is the familiar transformation rule for a tensor, with one factor of the Jacobian per index as usual. However, there is also an extra second term, meaning that the Christoffel symbols do not transform like a tensor.

(Similarly, you are also not supposed to raise or lower indices in the Christoffel symbols, but in practice, you can do that as long as you make it clear that it's just an abuse of notation - you are only adding factors of the metric, not creating a new tensor representation with different transformation properties.)

Due to the extra transformation term, the tensor object we calculated manually above using `TCalc` **must not be used**. Instead, we should use the built-in module `TChristoffel`, which not only performs the calculation automatically for us, but also knows that it has special transformation properties:

In[*]:=

Out[*]=

? TChristoffel

Symbol

TChristoffel[**metricID**] returns a reference to the Christoffel symbols (the coefficients of the Levi-Civita connection) calculated from the metric **metricID**. If the Christoffel symbols have not been calculated yet, they will be calculated and cached.

Let us, then, calculate the Christoffel symbols for the Schwarzschild metric properly, using **TChristoffel**:

In[*]:=

TList@TChristoffel["Schwarzschild"]

OGRe:

Schwarzschild->Christoffel:

$$\begin{aligned}
 \Gamma_{tr}^t &= \Gamma_{rt}^t = -\Gamma_{rr}^r &= \frac{M}{r(-2M+r)} \\
 \Gamma_{tt}^r & &= \frac{M(-2M+r)}{r^3} \\
 \Gamma_{\theta\theta}^r & &= 2M - r \\
 \Gamma_{\phi\phi}^r & &= (2M - r) \sin^2[\theta] \\
 \Gamma_{r\theta}^\theta &= \Gamma_{\theta r}^\theta = \Gamma_{r\phi}^\phi = \Gamma_{\phi r}^\phi &= \frac{1}{r} \\
 \Gamma_{\phi\phi}^\theta & &= -\cos[\theta] \sin[\theta] \\
 \Gamma_{\theta\phi}^\phi &= \Gamma_{\phi\theta}^\phi &= \cot[\theta]
 \end{aligned}$$

These are the same components we got before, but now they will transform properly. The calculated Christoffel symbols are **cached** internally inside the metric object on first use, and are subsequently reused whenever **TChristoffel["Schwarzschild"]** is called; this module does not create a separate tensor object with its own ID. This means that whenever we want to do any operations on the Christoffel symbols, including using them in arbitrary calculations with **TCalc**, we call **TChristoffel["Schwarzschild"]** again instead of using a specific ID string. The same is true for all other built-in tensors derived from the metric, which we will discuss below.

For maximal clarity, let us demonstrate the discrepancy in the coordinate transformation with a simple test metric:

In[]:=

```
TShow@TNewMetric["SimpleMetric", "Cartesian", DiagonalMatrix[{-x, 1, 1, 1}]]
```

OGRE:

$$\text{SimpleMetric: } g_{\mu\nu}(t, x, y, z) = \begin{pmatrix} -x & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

We calculate its Christoffel symbols in two ways. First manually, as we did above for the Schwarzschild metric:

In[]:=

```
TList@TChangeDefaultIndices[
  TCalc["SimpleMetricManualChristoffel",  $\frac{1}{2}$  "SimpleMetric"["λσ"].
    (TPartialD["μ"]."SimpleMetric"["νσ"] + TPartialD["ν"]."SimpleMetric"["σμ"] -
      TPartialD["σ"]."SimpleMetric"["μν"]), "r"], {1, -1, -1}]
```

OGRE:

SimpleMetricManualChristoffel:

$$\begin{aligned} \Gamma_{tx}^t &= \Gamma_{xt}^t = \frac{1}{2x} \\ \Gamma_{tt}^x &= \frac{1}{2} \end{aligned}$$

Then, with the built-in module `TChristoffel`:

In[]:=

```
TList@TChristoffel["SimpleMetric"]
```

OGRE:

SimpleMetric->Christoffel:

$$\begin{aligned} \Gamma_{tx}^t &= \Gamma_{xt}^t = \frac{1}{2x} \\ \Gamma_{tt}^x &= \frac{1}{2} \end{aligned}$$

The two results have the same components, as expected. But now, let us transform them to spherical coordinates. First, we transform the tensor object obtained using `TChristoffel`:

In[]:=

TList[TChristoffel["SimpleMetric"], "Spherical"]

SimpleMetric->Christoffel:

$$\begin{aligned}
\Gamma_{tr}^t &= \Gamma_{rt}^t &= \frac{1}{2r} \\
\Gamma_{t\theta}^t &= \Gamma_{\theta t}^t &= \frac{\text{Cot}[\theta]}{2} \\
\Gamma_{t\phi}^t &= \Gamma_{\phi t}^t &= -\frac{\text{Tan}[\phi]}{2} \\
\Gamma_{tt}^r & &= \frac{1}{2} \text{Cos}[\phi] \text{Sin}[\theta] \\
\Gamma_{\theta\theta}^r & &= -r \\
\Gamma_{\phi\phi}^r & &= -r \text{Sin}[\theta]^2 \\
\Gamma_{tt}^\theta & &= \frac{\text{Cos}[\theta] \text{Cos}[\phi]}{2r} \\
\Gamma_{r\theta}^\theta &= \Gamma_{\theta r}^\theta = \Gamma_{r\phi}^\phi = \Gamma_{\phi r}^\phi &= \frac{1}{r} \\
\Gamma_{\phi\phi}^\theta & &= -\text{Cos}[\theta] \text{Sin}[\theta] \\
\Gamma_{tt}^\phi & &= -\frac{\text{Csc}[\theta] \text{Sin}[\phi]}{2r} \\
\Gamma_{\theta\phi}^\phi &= \Gamma_{\phi\theta}^\phi &= \text{Cot}[\theta]
\end{aligned}$$

OGRe:

This is the **correct** representation of the Christoffel symbols in spherical coordinates, as it was recalculated directly from the metric in that coordinate system (OGRe does this instead of using the special transformation rule, as it turns out to be faster to just recalculate from scratch). However, if we transform the Christoffel symbols we obtained manually using **TCalc**, we get:

In[]:=

TList["SimpleMetricManualChristoffel", "Spherical"]

SimpleMetricManualChristoffel:

$$\begin{aligned}
\Gamma_{tr}^t &= \Gamma_{rt}^t &= \frac{1}{2r} \\
\Gamma_{t\theta}^t &= \Gamma_{\theta t}^t &= \frac{\text{Cot}[\theta]}{2} \\
\Gamma_{t\phi}^t &= \Gamma_{\phi t}^t &= -\frac{\text{Tan}[\phi]}{2} \\
\Gamma_{tt}^r & &= \frac{1}{2} \text{Cos}[\phi] \text{Sin}[\theta] \\
\Gamma_{tt}^\theta & &= \frac{\text{Cos}[\theta] \text{Cos}[\phi]}{2r} \\
\Gamma_{tt}^\phi & &= -\frac{\text{Csc}[\theta] \text{Sin}[\phi]}{2r}
\end{aligned}$$

OGRe:

This is **not** the correct result, since a tensor created manually with **TCalc** always transforms as an ordinary tensor, which is not the correct way to transform the Christoffel symbols. To verify that the

former result is indeed the correct one, let us change the default coordinate system of **SimpleMetric** to spherical:

```
In[ ]:= TChangeDefaultCoords["SimpleMetric", "Spherical"];
```

Now, when we calculate the Christoffel symbols manually from this metric, we will get their correct representation in spherical coordinates. This is because **TCalc** always performs the calculations internally in the default coordinates of the first tensor, so the result was calculated **from scratch** in spherical coordinates, instead of being calculated first in Cartesian coordinates and then transformed:

```
In[ ]:= TList@TChangeDefaultIndices[
  TCalc["SimpleMetricManualChristoffelSpherical",  $\frac{1}{2}$  "SimpleMetric"["λσ"] .
    (TPartialD["μ"] . "SimpleMetric"["νσ"] + TPartialD["ν"] . "SimpleMetric"["σμ"] -
      TPartialD["σ"] . "SimpleMetric"["μν"]), "r"], {1, -1, -1}]
```

SimpleMetricManualChristoffelSpherical:

$$\begin{aligned}
 \Gamma_{tr}^t &= \Gamma_{rt}^t &= \frac{1}{2r} \\
 \Gamma_{t\theta}^t &= \Gamma_{\theta t}^t &= \frac{\text{Cot}[\theta]}{2} \\
 \Gamma_{t\phi}^t &= \Gamma_{\phi t}^t &= -\frac{\text{Tan}[\phi]}{2} \\
 \Gamma_{tt}^r & &= \frac{1}{2} \text{Cos}[\phi] \text{Sin}[\theta] \\
 \Gamma_{\theta\theta}^r & &= -r \\
 \Gamma_{\phi\phi}^r & &= -r \text{Sin}[\theta]^2 \\
 \Gamma_{tt}^\theta & &= \frac{\text{Cos}[\theta] \text{Cos}[\phi]}{2r} \\
 \Gamma_{r\theta}^\theta &= \Gamma_{\theta r}^\theta = \Gamma_{r\phi}^\phi = \Gamma_{\phi r}^\phi &= \frac{1}{r} \\
 \Gamma_{\phi\phi}^\theta & &= -\text{Cos}[\theta] \text{Sin}[\theta] \\
 \Gamma_{tt}^\phi & &= -\frac{\text{Csc}[\theta] \text{Sin}[\phi]}{2r} \\
 \Gamma_{\theta\phi}^\phi &= \Gamma_{\phi\theta}^\phi &= \text{Cot}[\theta]
 \end{aligned}$$

Indeed, this is the exact same result we got when we transformed **TChristoffel["SimpleMetric"]** to spherical coordinates. We have learned an important lesson: since the Christoffel symbols do not transform like a tensor, we should always use the built-in module **TChristoffel** to calculate them, which ensures that they transform properly. (Of course, this method is also much more convenient than writing the explicit definition...)

For future use, let us define the **Friedmann-Lemaitre-Robertson-Walker (FLRW) metric**, which describes an expanding universe:

```
In[ ]:= TSetReservedSymbols[{a, k}];
TShow[TNewMetric["FLRW", "Spherical",
  DiagonalMatrix[{ -1,  $\frac{a[t]^2}{1 - k r^2}$ ,  $a[t]^2 r^2$ ,  $a[t]^2 r^2 \sin[\theta]^2$  }]]]
```

OGRe:

$$\text{FLRW: } g_{\mu\nu}(t, r, \theta, \phi) = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & \frac{a^2}{1 - k r^2} & 0 & 0 \\ 0 & 0 & a^2 r^2 & 0 \\ 0 & 0 & 0 & a^2 r^2 \sin[\theta]^2 \end{pmatrix}$$

This metric has the line element:

```
In[ ]:= TLineElement["FLRW"]
```

Out[]:=

$$-dt^2 + a[t]^2 \left(\frac{dr^2}{1 - k r^2} + r^2 d\theta^2 + r^2 d\phi^2 \sin[\theta]^2 \right)$$

and the volume element:

```
In[ ]:= TVolumeElement["FLRW"]
```

Out[]:=

$$r^2 \text{Abs}\left[\frac{a[t]^3 \sin[\theta]}{\sqrt{-1 + k r^2}}\right] dt \wedge dr \wedge d\theta \wedge d\phi$$

Here, $a(t)$ is the **scale factor** and k is the curvature of the spatial surfaces, with $k = +1, 0, -1$ corresponding to positively curved, flat, or negatively curved respectively. Its Christoffel symbols can be easily calculated using **TChristoffel**:

In[]:=

TList@TChristoffel["FLRW"]

FLRW→Christoffel:

$$\begin{aligned}
 \Gamma_{rr}^t &= \frac{a \partial_t a}{1 - k r^2} \\
 \Gamma_{\theta\theta}^t &= a r^2 \partial_t a \\
 \Gamma_{\phi\phi}^t &= a r^2 \partial_t a \sin[\theta]^2 \\
 \Gamma_{tr}^r = \Gamma_{rt}^r = \Gamma_{t\theta}^\theta = \Gamma_{\theta t}^\theta = \Gamma_{t\phi}^\phi = \Gamma_{\phi t}^\phi &= \frac{\partial_t a}{a} \\
 \Gamma_{rr}^r &= \frac{k r}{1 - k r^2} \\
 \Gamma_{\theta\theta}^r &= r(-1 + k r^2) \\
 \Gamma_{\phi\phi}^r &= r(-1 + k r^2) \sin[\theta]^2 \\
 \Gamma_{r\theta}^\theta = \Gamma_{\theta r}^\theta = \Gamma_{r\phi}^\phi = \Gamma_{\phi r}^\phi &= \frac{1}{r} \\
 \Gamma_{\phi\phi}^\theta &= -\cos[\theta] \sin[\theta] \\
 \Gamma_{\theta\phi}^\phi = \Gamma_{\phi\theta}^\phi &= \cot[\theta]
 \end{aligned}$$

OGRE:

Notice how the derivatives of the function **a** are shown by **TList** in the notation $\partial_t a$ instead of $a'[t]$. This may not seem like much of an improvement in this case, but in the case of partial derivatives of functions of many arguments, it can greatly improve readability. Here is a simple example:

In[]:=

**TShow@TNewTensor["PartialDerivatives", "FLRW", "Cartesian",
 {1}, { $\partial_x f[x, y, z]$, $\partial_{x,y} f[x, y, z]$, $\partial_{x,y,z} f[x, y, z]$, $\partial_{x,y,\{z,2\}} f[x, y, z]$ }]**

$$\text{PartialDerivatives: } \square^\mu(t, x, y, z) = \begin{pmatrix} \partial_x f \\ \partial_x \partial_y f \\ \partial_x \partial_y \partial_z f \\ \partial_x \partial_y \partial_z^2 f \end{pmatrix}$$

OGRE:

Compare this with the raw components, where the order of the derivative with respect to each variable is determined by position, and the $[x, y, z]$ function argument is shown explicitly:

In[]:=

TGetComponents["PartialDerivatives"]

... **TGetComponents**: Using the default index configuration {1} and the default coordinate system "Cartesian".

Out[]:=

 $\{f^{(1,0,0)}[x, y, z], f^{(1,1,0)}[x, y, z], f^{(1,1,1)}[x, y, z], f^{(1,1,2)}[x, y, z]\}$

The Riemann tensor

The **Riemann curvature tensor** $R^\rho_{\sigma\mu\nu}$ can be calculated from the Christoffel symbols using the definition:

$$R^\rho_{\sigma\mu\nu} = \partial_\mu \Gamma^\rho_{\nu\sigma} - \partial_\nu \Gamma^\rho_{\mu\sigma} + \Gamma^\rho_{\mu\lambda} \Gamma^\lambda_{\nu\sigma} - \Gamma^\rho_{\nu\lambda} \Gamma^\lambda_{\mu\sigma}.$$

Even though it contains partial derivatives, it nonetheless transforms like a tensor under a change of coordinates, because the extra transformation terms exactly cancel each other. To calculate this tensor, we can simply write down the formula in **TCalc** with the correct indices contracted. Note that this time we specified the LHS indices explicitly, since they are not in the same order as the RHS indices in our definition:

In[*]:=

```
TList@TChangeDefaultIndices[TCalc["SchwarzschildManualRiemann"]["ρσμν"],
  TPartialD["μ"].TChristoffel["Schwarzschild"]["ρνσ"] -
  TPartialD["ν"].TChristoffel["Schwarzschild"]["ρμσ"] +
  TChristoffel["Schwarzschild"]["ρμλ"].TChristoffel["Schwarzschild"]["λνσ"] -
  TChristoffel["Schwarzschild"]["ρνλ"].
  TChristoffel["Schwarzschild"]["λμσ"], "R"], {1, -1, -1, -1}]
```

OGRE:

SchwarzschildManualRiemann:

$$\begin{aligned} R^t_{rtr} &= -R^t_{rrt} &= \frac{2M}{r^2(-2M+r)} \\ R^t_{\theta t\theta} &= R^r_{\theta r\theta} = -R^t_{\theta\theta t} = -R^r_{\theta\theta r} &= -\frac{M}{r} \\ R^t_{\phi t\phi} &= R^r_{\phi r\phi} = -R^t_{\phi\phi t} = -R^r_{\phi\phi r} &= -\frac{M \sin^2[\theta]}{r} \\ R^r_{ttr} &= -R^r_{trt} &= \frac{2M(-2M+r)}{r^4} \\ R^\theta_{tt\theta} &= R^\phi_{tt\phi} = -R^\theta_{t\theta t} = -R^\phi_{t\phi t} &= \frac{M(2M-r)}{r^4} \\ R^\theta_{rr\theta} &= R^\phi_{rr\phi} = -R^\theta_{r\theta r} = -R^\phi_{r\phi r} &= \frac{M}{r^2(-2M+r)} \\ R^\theta_{\phi\theta\phi} &= -R^\theta_{\phi\phi\theta} &= \frac{2M \sin^2[\theta]}{r} \\ R^\phi_{\theta\theta\phi} &= -R^\phi_{\theta\phi\theta} &= -\frac{2M}{r} \end{aligned}$$

The Riemann tensor with all its indices lowered satisfies the following symmetry and anti-symmetry relations:

$$R_{\rho\sigma\mu\nu} = -R_{\sigma\rho\mu\nu} = -R_{\rho\sigma\nu\mu} = R_{\mu\nu\rho\sigma}.$$

We can verify this for the Schwarzschild Riemann tensor using **TList**, as it automatically detects components that are the same up to sign:

In[]:= **TList[TRiemann["Schwarzschild"], {-1, -1, -1, -1}]**

OGRe:

Schwarzschild→Riemann:

$$\begin{aligned}
 R_{trtr} &= R_{trrt} = -R_{trrt} = -R_{rttr} &= -\frac{2M}{r^3} \\
 R_{t\theta t\theta} &= R_{\theta t\theta t} = -R_{t\theta\theta t} = -R_{\theta t t\theta} &= \frac{M(-2M+r)}{r^2} \\
 R_{t\phi t\phi} &= R_{\phi t\phi t} = -R_{t\phi\phi t} = -R_{\phi t t\phi} &= \frac{M(-2M+r)\sin[\theta]^2}{r^2} \\
 R_{r\theta r\theta} &= R_{\theta r\theta r} = -R_{r\theta\theta r} = -R_{\theta r r\theta} &= \frac{M}{2M-r} \\
 R_{r\phi r\phi} &= R_{\phi r\phi r} = -R_{r\phi\phi r} = -R_{\phi r r\phi} &= \frac{M\sin[\theta]^2}{2M-r} \\
 R_{\theta\phi\theta\phi} &= R_{\phi\theta\phi\theta} = -R_{\theta\phi\phi\theta} = -R_{\phi\theta\theta\phi} &= 2Mr\sin[\theta]^2
 \end{aligned}$$

Don't worry - you don't need to write the explicit definition of the Riemann tensor every time you want to calculate it. Instead, OGRE offers the built-in module **TRiemann**:

In[]:= **? TRiemann**

Out[]:=

Symbol

TRiemann[**metricID**] returns a reference to the Riemann tensor calculated from the metric **metricID**. If the Riemann tensor has not been calculated yet, it will be calculated and cached.

For example, for the FLRW metric we get:

In[]:= **TList@TRiemann["FLRW"]**

OGRe:

FLRW→Riemann:

$$\begin{aligned}
 R^t_{rtr} &= -R^t_{rrt} &= \frac{a\partial_t^2 a}{1-kr^2} \\
 R^t_{\theta t\theta} &= -R^t_{\theta\theta t} &= ar^2\partial_t^2 a \\
 R^t_{\phi t\phi} &= -R^t_{\phi\phi t} &= ar^2\partial_t^2 a\sin[\theta]^2 \\
 R^r_{ttr} &= R^\theta_{tt\theta} = R^\phi_{tt\phi} = -R^r_{trt} = -R^\theta_{t\theta t} = -R^\phi_{t\phi t} &= \frac{\partial_t^2 a}{a} \\
 R^r_{\theta r\theta} &= R^\phi_{\theta\phi\theta} = -R^r_{\theta\theta r} = -R^\phi_{\theta\theta\phi} &= r^2(k + (\partial_t a)^2) \\
 R^r_{\phi r\phi} &= R^\theta_{\phi\theta\phi} = -R^r_{\phi\phi r} = -R^\theta_{\phi\phi\theta} &= r^2(k + (\partial_t a)^2)\sin[\theta]^2 \\
 R^\theta_{rr\theta} &= R^\phi_{rr\phi} = -R^\theta_{r\theta r} = -R^\phi_{r\phi r} &= \frac{k + (\partial_t a)^2}{-1 + kr^2}
 \end{aligned}$$

Like **TChristoffel**, and like all the other built-in modules for calculating derived tensors we will discuss below, **TRiemann** calculates and caches the Riemann tensor the first time it is called, and then reuses the cached version on subsequent calls.

Using **TRiemann** also has the advantage that it takes a metric as an input, and will automatically calculate and cache the Christoffel symbols of the metric (as if using **TChristoffel**) if they have not already been calculated. The same principle also applies to the other built-in modules for calculating curvature tensors, which we will present below; they always take a metric as input, and will calculate any intermediate tensors in their definitions automatically as needed.

The Ricci tensor and scalar

The **Ricci tensor** $R_{\mu\nu}$ is the trace of the first and third indices of the Riemann tensor:

$$R_{\mu\nu} = R^{\lambda}{}_{\mu\lambda\nu}.$$

Therefore, we can calculate it by taking the trace, with the usual **TCalc** syntax. For the Schwarzschild metric, the Ricci tensor vanishes:

```
In[ ]:= TList@TCalc[TRiemann["Schwarzschild"] ["λμλν"], "R"]
```

Result:
No non-zero elements.

We can also use the shorthand module **TRicci**:

```
In[ ]:= ? TRicci
```

Out[]:=

Symbol

TRicci[metricID] returns a reference to the Ricci tensor calculated from the metric **metricID**. If the Ricci tensor has not been calculated yet, it will be calculated and cached.

▼

Here is the Ricci tensor for the FLRW metric:

In[]:= **TList@TRicci**["FLRW"]

OGRe:

FLRW→Ricci:

$$R_{tt} = -\frac{3\partial_t^2 a}{a}$$

$$R_{rr} = \frac{a\partial_t^2 a + 2(k + (\partial_t a)^2)}{1 - kr^2}$$

$$R_{\theta\theta} = r^2 (a\partial_t^2 a + 2(k + (\partial_t a)^2))$$

$$R_{\phi\phi} = r^2 (a\partial_t^2 a + 2(k + (\partial_t a)^2)) \sin[\theta]^2$$

The **Ricci scalar** is the trace of the Ricci tensor:

$$R = R^\lambda{}_\lambda = g^{\mu\nu} R_{\mu\nu}.$$

We can calculate it from the Ricci tensor by taking the trace:

In[]:= **TShow@TCalc**[**TRicci**["FLRW"] ["μμ"], "R"]

OGRe:

$$\text{Result: } R(t, r, \theta, \phi) = \frac{6(k + a\partial_t^2 a + (\partial_t a)^2)}{a^2}$$

Or, as usual, we can simply use the shorthand module **TRicciScalar** to calculate it directly from the metric:

In[]:= **?TRicciScalar**

Out[]:=

Symbol

TRicciScalar[**metricID**] returns a reference to the Ricci scalar calculated from the metric **metricID**. If the Ricci scalar has not been calculated yet, it will be calculated and cached.

▼

In[]:= **TList@TRicciScalar**["FLRW"]

OGRe:

FLRW→RicciScalar:

$$R = \frac{6(k + a\partial_t^2 a + (\partial_t a)^2)}{a^2}$$

The Kretschmann scalar

The **Kretschmann scalar** is the norm-squared of the Riemann tensor:

$$K = R_{\rho\sigma\mu\nu} R^{\rho\sigma\mu\nu}.$$

It can be easily calculated in OGRE for the Schwarzschild metric using **TCalc** as follows:

```
In[ ]:= TShow@
TCalc[TRiemann["Schwarzschild"] ["ρσμν"].TRiemann["Schwarzschild"] ["ρσμν"], "K"]
```

OGRe: Result: $K(t, r, \theta, \phi) = \frac{48 M^2}{r^6}$

Even more easily, we can use the **TNormSquared** module on the Riemann tensor (but the result will be the scalar itself, not a new tensor object):

```
In[ ]:= TNormSquared[TRiemann["Schwarzschild"]]
```

Out[]:=

$$\frac{48 M^2}{r^6}$$

However, the easiest way to calculate it is using the built-in module **TKretschmann**:

```
In[ ]:= ?TKretschmann
```

Out[]:=

Symbol

TKretschmann[**metricID**] returns a reference to the Kretschmann scalar calculated from the metric **metricID**. If the Kretschmann scalar has not been calculated yet, it will be calculated and cached.



```
In[ ]:= TShow@TKretschmann["Schwarzschild"]
```

OGRe: Schwarzschild→Kretschmann: $K(t, r, \theta, \phi) = \frac{48 M^2}{r^6}$

For the FLRW metric, we get:

```
In[ ]:= TShow@TKretschmann["FLRW"]
```

OGRe: FLRW→Kretschmann: $K(t, r, \theta, \phi) = \frac{12 \left((k + (\partial_t a)^2)^2 + a^2 (\partial_t^2 a)^2 \right)}{a^4}$

The Einstein tensor

The **Einstein tensor** $G_{\mu\nu}$ is given by:

$$G_{\mu\nu} = R_{\mu\nu} - \frac{1}{2} g_{\mu\nu} R.$$

As with all other curvature tensors, we can calculate it by combining the previously calculated tensors with the usual syntax:

```
In[ ]:= TList[TCalc[TRicci["FLRW"] ["μν"] -  $\frac{1}{2}$  "FLRW" ["μν"].TRicciScalar["FLRW"] [""], "G"]]
```

Result:

$$G_{tt} = \frac{3(k+(\partial_t a)^2)}{a^2}$$

$$G_{rr} = \frac{k+2a\partial_t^2 a+(\partial_t a)^2}{-1+kr^2}$$

$$G_{\theta\theta} = -r^2(k+2a\partial_t^2 a+(\partial_t a)^2)$$

$$G_{\phi\phi} = -r^2(k+2a\partial_t^2 a+(\partial_t a)^2)\sin[\theta]^2$$

OGRe:

Or we can simply use the built-in module **TEinstein**:

```
In[ ]:= ?TEinstein
```

Out[]:=

Symbol

TEinstein[**metricID**] returns a reference to the Einstein tensor calculated from the metric **metricID**. If the Einstein tensor has not been calculated yet, it will be calculated and cached.

▼

```
In[ ]:= TList@TEinstein["FLRW"]
```

FLRW→Einstein:

$$G_{tt} = \frac{3(k+(\partial_t a)^2)}{a^2}$$

$$G_{rr} = \frac{k+2a\partial_t^2 a+(\partial_t a)^2}{-1+kr^2}$$

$$G_{\theta\theta} = -r^2(k+2a\partial_t^2 a+(\partial_t a)^2)$$

$$G_{\phi\phi} = -r^2(k+2a\partial_t^2 a+(\partial_t a)^2)\sin[\theta]^2$$

OGRe:

The Weyl tensor

The **Weyl tensor** $C^{\rho}{}_{\sigma\mu\nu}$ is the trace-free part of the Riemann tensor, given by:

$$C^{\rho}{}_{\sigma\mu\nu} = R^{\rho}{}_{\sigma\mu\nu} - \frac{1}{n-2} (\delta^{\rho}{}_{\mu} R_{\nu\sigma} - \delta^{\rho}{}_{\nu} R_{\mu\sigma} - g_{\sigma\mu} R^{\rho}{}_{\nu} + g_{\sigma\nu} R^{\rho}{}_{\mu}) + \frac{R}{(n-1)(n-2)} (\delta^{\rho}{}_{\mu} g_{\nu\sigma} - \delta^{\rho}{}_{\nu} g_{\mu\sigma}),$$

where n is the dimension. In dimensions $n \leq 3$, the Weyl tensor vanishes identically. We can calculate it in OGRE using the built-in module **TWeyl**:

In[*]:=

Out[*]=

? TWeyl

Symbol

TWeyl[metricID] returns a reference to the Weyl tensor calculated from the metric **metricID**. If the Weyl tensor has not been calculated yet, it will be calculated and cached.

The FLRW metric is conformally flat, so its Weyl tensor vanishes:

In[*]:=

TList@TWeyl["FLRW"]

OGRe:

FLRW->Weyl:
No non-zero elements.

Since the Schwarzschild metric is Ricci-flat, the Weyl tensor is equal to the Riemann tensor, as we can verify by taking the difference of the two tensors:

In[*]:=

TList@TCalc[TWeyl["Schwarzschild"] ["ρσμν"] - TRiemann["Schwarzschild"] ["ρσμν"]]

OGRe:

Result:
No non-zero elements.

For an example where the Weyl tensor is non-zero but differs from the Riemann tensor, let us define the **Schwarzschild-de Sitter metric**:

In[]:=

```
TSetReservedSymbols[Λ];
TShow[TNewMetric["SchwarzschildDeSitter", "Spherical",
  DiagonalMatrix[{-(1 - 2 M/r - Λ r^2/3), 1/(1 - 2 M/r - Λ r^2/3), r^2, r^2 Sin[θ]^2}]]]
```

OGRE:

 SchwarzschildDeSitter: $g_{\mu\nu}$

$$)= \begin{pmatrix} -1 + \frac{2M}{r} + \frac{r^2 \Lambda}{3} & 0 & 0 \\ 0 & -\frac{3r}{6M - 3r + r^3 \Lambda} & 0 \\ 0 & 0 & r^2 \\ 0 & 0 & 0 \end{pmatrix}$$

Its line element is:

In[]:=

```
TLineElement["SchwarzschildDeSitter"]
```

Out[]:=

$$dt^2 \left(-1 + \frac{2M}{r} + \frac{r^2 \Lambda}{3} \right) + r^2 \left(d\theta^2 - \frac{3 dr^2}{6M - 3r + r^3 \Lambda} \right) + r^2 d\phi^2 \sin^2[\theta]$$

Calculating the Weyl tensor, we find:

In[]:=

```
TList@TWeyl["SchwarzschildDeSitter"]
```

OGRE:

SchwarzschildDeSitter->Weyl:

$$\begin{aligned} C^t_{rtr} &= -C^t_{rrt} &= -\frac{6M}{r^2(6M - 3r + r^3 \Lambda)} \\ C^t_{\theta t \theta} &= C^r_{\theta r \theta} = -C^t_{\theta \theta t} = -C^r_{\theta \theta r} &= -\frac{M}{r} \\ C^t_{\phi t \phi} &= C^r_{\phi r \phi} = -C^t_{\phi \phi t} = -C^r_{\phi \phi r} &= -\frac{M \sin^2[\theta]}{r} \\ C^r_{ttr} &= -C^r_{trt} &= -\frac{2M(6M - 3r + r^3 \Lambda)}{3r^4} \\ C^\theta_{tt\theta} &= C^\phi_{tt\phi} = -C^\theta_{t\theta t} = -C^\phi_{t\phi t} &= \frac{M(6M - 3r + r^3 \Lambda)}{3r^4} \\ C^\theta_{rr\theta} &= C^\phi_{rr\phi} = -C^\theta_{r\theta r} = -C^\phi_{r\phi r} &= -\frac{3M}{r^2(6M - 3r + r^3 \Lambda)} \\ C^\theta_{\phi\theta\phi} &= -C^\theta_{\phi\phi\theta} &= \frac{2M \sin^2[\theta]}{r} \\ C^\phi_{\theta\theta\phi} &= -C^\phi_{\theta\phi\theta} &= -\frac{2M}{r} \end{aligned}$$

To see explicitly how it differs from the Riemann tensor, compare the Kretschmann scalar:

```
In[ ]:= TShow@TKretschmann["SchwarzschildDeSitter"]
```

```
OGRe:
```

$$\text{SchwarzschildDeSitter} \rightarrow \text{Kretschmann}: K(t, r, \theta, \phi) = \frac{48 M^2}{r^6} + \frac{8 \Lambda^2}{3}$$

With the norm-squared of the Weyl tensor:

```
In[ ]:= TNormSquared[TWeyl["SchwarzschildDeSitter"]]
```

```
Out[ ]:=
```

$$\frac{48 M^2}{r^6}$$

The Kretschmann scalar contains both the Schwarzschild mass term and the cosmological constant term, while the norm-squared of the Weyl tensor contains only the mass term.

Covariant derivatives

The partial derivative has limited use in general relativity, as **it does not transform like a tensor**. Therefore, it is only used in special cases, such as calculating the Christoffel symbols and the Riemann tensor. The **covariant derivative** ∇_μ is a generalization of the partial derivative, which does transform like a tensor (as long as it acts on a proper tensor). It is defined as follows:

- On a scalar Φ , the covariant derivative acts as $\nabla_\mu \Phi \equiv \partial_\mu \Phi$.
- On a vector v^μ , the covariant derivative acts as $\nabla_\mu v^\nu \equiv \partial_\mu v^\nu + \Gamma_{\mu\lambda}^\nu v^\lambda$.
- On a covector w_μ , the covariant derivative acts as $\nabla_\mu w_\nu \equiv \partial_\mu w_\nu - \Gamma_{\mu\nu}^\lambda w_\lambda$.

More generally, on a rank (p, q) tensor with components $T_{\sigma_1 \dots \sigma_q}^{\nu_1 \dots \nu_p}$, the covariant derivative $\nabla_\mu T_{\sigma_1 \dots \sigma_q}^{\nu_1 \dots \nu_p}$ is defined as follows:

- The first term will be $\partial_\mu T_{\sigma_1 \dots \sigma_q}^{\nu_1 \dots \nu_p}$.
- We **add** one term $\Gamma_{\mu\lambda}^{\nu_i} T_{\sigma_1 \dots \sigma_q}^{\nu_1 \dots \lambda \dots \nu_p}$ for each upper index ν_i .
- We **subtract** one term $\Gamma_{\mu\sigma_i}^\lambda T_{\sigma_1 \dots \lambda \dots \sigma_q}^{\nu_1 \dots \nu_p}$ for each lower index σ_i .

Note that even though the covariant derivative is made from ingredients that do not transform like tensors - the partial derivative and the Christoffel symbols - the unwanted terms in the transformations of these ingredients cancel each other exactly, so that in the end, the entire sum does transform like a tensor.

As usual, we can, of course, write down the covariant derivative manually. For example, the covariant divergence of the metric is:

$$\nabla_\mu g_{\alpha\beta} = \partial_\mu g_{\alpha\beta} - \Gamma_{\mu\alpha}^\lambda g_{\lambda\beta} - \Gamma_{\mu\beta}^\lambda g_{\alpha\lambda}.$$

It should vanish, by definition, for any metric; this is what we meant when we said the Levi-Civita connection **preserves** the metric. Indeed, we have for the Schwarzschild metric:

In[]:=

```
TList@TCalc[TPartialD["μ"]."Schwarzschild"["αβ"] -
  TChristoffel["Schwarzschild"] ["λμα"]."Schwarzschild"["λβ"] -
  TChristoffel["Schwarzschild"] ["λμβ"]."Schwarzschild"["αλ"]]
```

OGRE:

Result:
No non-zero elements.

Much more conveniently, the covariant derivative is represented in OGRE as **TCovariantD**. It will automatically add the correct terms, as detailed above, for each of the tensor's indices. To use it, simply contract it with any tensor, just like **TPartialD**:

In[]:=

```
? TCovariantD
```

Out[]:=

Symbol

TCovariantD[**index**] represents the covariant derivative when used in a tensor expression given to TCalc[]. The Levi-Civita connection associated with the tensor's metric will be obtained using TChristoffel[].

▼

For example, we can check that the covariant derivative of the FLRW metric also vanishes:

In[]:=

```
TList@TCalc[TCovariantD["μ"]."FLRW"["αβ"]]
```

OGRE:

Result:
No non-zero elements.

The covariant divergence of the Einstein tensor is:

$$\nabla_{\mu} G^{\mu\nu} = \partial_{\mu} G^{\mu\nu} + \Gamma_{\mu\lambda}^{\mu} G^{\lambda\nu} + \Gamma_{\mu\lambda}^{\nu} G^{\mu\lambda}.$$

Note that it involves a contraction in the index μ , which becomes a trace in the first Christoffel symbol. This expression vanishes because of the **Bianchi identity**:

$$\nabla_{\mu} R^{\mu\nu} = \frac{1}{2} \nabla^{\nu} R \Rightarrow \nabla_{\mu} G^{\mu\nu} = 0.$$

To calculate it in OGRE, we simply write:

In[]:=

```
TList@TCalc[TCovariantD["μ"].TEinstein["FLRW"] ["μν"]]
```

OGRE:

Result:
No non-zero elements.

Finally, for a non-trivial result, let us recall that the stress-energy tensor should be **conserved**:

$$\nabla_{\mu} T^{\mu\nu} = \partial_{\mu} T^{\mu\nu} + \Gamma_{\mu\lambda}^{\mu} T^{\lambda\nu} + \Gamma_{\mu\lambda}^{\nu} T^{\mu\lambda} = 0.$$

This follows from the fact that $\nabla_\mu G^{\mu\nu} = 0$, combined with the **Einstein equation**:

$$G_{\mu\nu} = \kappa T_{\mu\nu},$$

where $\kappa = 1$ or $\kappa = 8\pi$ depending on your preferred units. However, unlike $\nabla_\mu G^{\mu\nu} = 0$, the relation $\nabla_\mu T^{\mu\nu} = 0$ is **not** an identity; it is an **energy-momentum conservation equation**. To derive the equation for the FLRW metric, let us first define the rest-frame fluid 4-velocity in this spacetime:

In[]:=

```
TShow@TNewTensor["RestVelocity", "FLRW", "Spherical", {1}, {1, 0, 0, 0}, "u"]
```

OGRe:

$$\text{RestVelocity: } u^\mu(t, r, \theta, \phi) = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Using the 4-velocity and the metric, we redefine the perfect fluid stress tensor in the FLRW spacetime using the formula $T^{\mu\nu} = (\rho + p) u^\mu u^\nu + p g^{\mu\nu}$, and give ρ and p spacetime dependence:

In[]:=

```
TCalc["PerfectFluidFLRW",  
  (\rho[t, r, \theta, \phi] + p[t, r, \theta, \phi]) "RestVelocity"["\mu"]."RestVelocity"["\nu"] +  
  p[t, r, \theta, \phi] "FLRW"["\mu\nu"], "T");  
TChangeDefaultIndices["PerfectFluidFLRW", {1, 1}];  
TShow["PerfectFluidFLRW"]
```

OGRe:

$$\text{PerfectFluidFLRW: } T^{\mu\nu}(t, r, \theta, \phi) = \begin{pmatrix} \rho & 0 & 0 & 0 \\ 0 & \frac{p(1-k r^2)}{a^2} & 0 & 0 \\ 0 & 0 & \frac{p}{a^2 r^2} & 0 \\ 0 & 0 & 0 & \frac{p \text{Csc}[\theta]^2}{a^2 r^2} \end{pmatrix}$$

Finally, we take the covariant derivative of the stress tensor:

In[]:=

```
TList@TCalc["FLRWConservation", TCovariantD["\mu"]."PerfectFluidFLRW"["\mu\nu"]]
```

OGRe:

FLRWConservation:

$$\square^t = \frac{3(p+\rho)\partial_t a}{a} + \partial_t \rho$$

$$\square^r = \frac{(1-k r^2)\partial_r \rho}{a^2}$$

$$\square^\theta = \frac{\partial_\theta \rho}{a^2 r^2}$$

$$\square^\phi = \frac{\text{Csc}[\theta]^2 \partial_\phi \rho}{a^2 r^2}$$

From demanding that the t component vanishes, we get the following equation:

$$\dot{\rho} = -3(\rho + p) \frac{\dot{a}}{a}.$$

We see that in an expanding universe, energy is not conserved, but rather, the energy density changes with time in a way that depends on the scale factor. If the universe is not expanding, that is, $\dot{a} = 0$, then energy will be conserved.

Overwriting metrics

If overwriting tensors has been allowed using `TSetAllowOverwrite[True]`, and we overwrite a metric tensor, then all derived tensors cached inside that metric will be automatically deleted, since the corresponding quantities for the new metric will generally be different. For example, let us overwrite the FLRW metric with a similar metric where k is equal to 0:

In[]:=

```
currentOverwrite = TSetAllowOverwrite[];
TSetAllowOverwrite[True];
TShow@TNewMetric["FLRW", "Spherical",
  DiagonalMatrix[{-1, a[t]^2, a[t]^2 r^2, a[t]^2 r^2 Sin[θ]^2}]]
```

... **TSetAllowOverwrite**: Overwriting tensors turned on.

... **TMessage**: Overwriting the tensor "FLRW".

TNewMetric: All built-in derived tensors previously calculated and cached in the metric being overwritten have been deleted.

OGRe:

$$\text{FLRW: } g_{\mu\nu}(t, r, \theta, \phi) = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & a^2 & 0 & 0 \\ 0 & 0 & a^2 r^2 & 0 \\ 0 & 0 & 0 & a^2 r^2 \sin^2[\theta] \end{pmatrix}$$

If we now try to access, for example, `TChristoffel["FLRW"]`, OGRE will recalculate it automatically from the new metric:

```
In[ ]:= TList@TChristoffel["FLRW"]
```

FLRW→Christoffel:

$$\begin{aligned}
 \Gamma^t_{rr} &= a \partial_t a \\
 \Gamma^t_{\theta\theta} &= a r^2 \partial_t a \\
 \Gamma^t_{\phi\phi} &= a r^2 \partial_t a \sin^2[\theta] \\
 \Gamma^r_{tr} = \Gamma^r_{rt} = \Gamma^\theta_{t\theta} = \Gamma^\theta_{\theta t} = \Gamma^\phi_{t\phi} = \Gamma^\phi_{\phi t} &= \frac{\partial_t a}{a} \\
 \Gamma^r_{\theta\theta} &= -r \\
 \Gamma^r_{\phi\phi} &= -r \sin^2[\theta] \\
 \Gamma^\theta_{r\theta} = \Gamma^\theta_{\theta r} = \Gamma^\phi_{r\phi} = \Gamma^\phi_{\phi r} &= \frac{1}{r} \\
 \Gamma^\theta_{\phi\phi} &= -\cos[\theta] \sin[\theta] \\
 \Gamma^\phi_{\theta\phi} = \Gamma^\phi_{\phi\theta} &= \cot[\theta]
 \end{aligned}$$

OGRe:

The same applies to all other tensors derived from the metric using built-in modules. However, user-defined tensors created with `TNewTensor` that are associated to the metric are not automatically deleted, since they do not depend on the metric itself.

Finally, let us reset `TSetAllowOverwrite` back to its previous setting:

```
In[ ]:= Off[TSetAllowOverwrite::Notify];
TSetAllowOverwrite[currentOverwrite];
On[TSetAllowOverwrite::Notify]
```

Curves and geodesics

The curve Lagrangian

Consider a **curve**, which is a function $x^\mu(\lambda)$ on the manifold where λ is called the **curve parameter**. The **curve Lagrangian** of a metric is defined as the norm-squared of the tangent to the curve:

$$L = g_{\mu\nu} \dot{x}^\mu \dot{x}^\nu,$$

where $\dot{x}^\mu$ is the first derivative of x^μ with respect to the curve parameter (in Newton dot notation). We can calculate it using the module `TLagrangian`:

In[]:=
Out[]:=

? TLagrangian

Symbol

TLagrangian[**metricID**] returns a reference to the curve Lagrangian of the metric **metricID**, defined as the norm-squared of the tangent to the curve. If the Lagrangian has not been calculated yet, it will be calculated and cached.

Taking the square root of (the absolute value of) the Lagrangian yields the integrand of the curve length functional. Varying the Lagrangian using the Euler-Lagrange equations yields the geodesic equations.

The result will be given in terms of the coordinate symbols as functions of the curve parameter and their derivatives with respect to the curve parameter. The curve parameter can be selected using TSetCurveParameter[].

For example:

In[]:=

TList@TLagrangian["Minkowski"]

OGRe:

Minkowski->Lagrangian:

$$L = -\dot{t}^2 + \dot{x}^2 + \dot{y}^2 + \dot{z}^2$$

In[]:=

TList@TLagrangian["Schwarzschild"]

OGRe:

Schwarzschild->Lagrangian:

$$L = \frac{\dot{r}^2}{1 - \frac{2M}{r}} + \left(-1 + \frac{2M}{r}\right) \dot{t}^2 + r^2 \left(\dot{\theta}^2 + \dot{\phi}^2 \sin^2[\theta]\right)$$

In[]:=

TList@TLagrangian["FLRW"]

OGRe:

FLRW->Lagrangian:

$$L = -\dot{t}^2 + a^2 \left(\dot{r}^2 + r^2 \left(\dot{\theta}^2 + \dot{\phi}^2 \sin^2[\theta]\right)\right)$$

In[]:=

TList@TLagrangian["Alcubierre"]

OGRe:

Alcubierre->Lagrangian:

$$L = \left(-1 + f^2 v^2\right) \dot{t}^2 + \dot{x}^2 + \dot{y}^2 - 2 f v \dot{t} \dot{z} + \dot{z}^2$$

Notice how TList (and TShow) use Newton dot notation for the derivatives of the coordinate functions, for improved readability. To get the full expressions with the explicit derivatives, we can use TGetComponents. For example:

```
In[ ]:= TGetComponents@TLagrangian["Minkowski"]
```

... **TGetComponents**: Using the default index configuration {} and the default coordinate system "Cartesian".

```
Out[ ]:=
```

$$\{-\mathbf{t}'[\lambda]^2 + \mathbf{x}'[\lambda]^2 + \mathbf{y}'[\lambda]^2 + \mathbf{z}'[\lambda]^2\}$$

Geodesic equations from the Lagrangian

By applying the **Euler-Lagrange equations** to the curve Lagrangian:

$$\frac{d}{d\lambda} \left(\frac{\partial L}{\partial \dot{x}^\mu} \right) - \frac{\partial L}{\partial x^\mu} = 0,$$

we can obtain the geodesic equations for our spacetime. This is done using the module **TGeodesicFromLagrangian**:

```
In[ ]:= ? TGeodesicFromLagrangian
```

```
Out[ ]:=
```

Symbol

TGeodesicFromLagrangian[metricID] returns a reference to the geodesic equations obtained using the curve Lagrangian of the metric **metricID**. Equating the components to zero will yield the full system of geodesic equations.

Derivatives with respect to the curve parameter in the Euler–Lagrange equation will be left unevaluated using **Inactive[]**, which can sometimes help solve the geodesic equations by inspection. Use **Activate[]** to evaluate the derivatives.

The result will be given in terms of the coordinate symbols as functions of the curve parameter and their derivatives with respect to the curve parameter. The curve parameter can be selected using **TSetCurveParameter[]**.

For the Minkowski metric, the geodesic equations are:

```
In[ ]:= TList@TGeodesicFromLagrangian["Minkowski"]
```

OGRe:

Minkowski→GeodesicFromLagrangian:

$$0_t = \partial_\lambda(-\dot{t})$$

$$0_x = \partial_\lambda \dot{x}$$

$$0_y = \partial_\lambda \dot{y}$$

$$0_z = \partial_\lambda \dot{z}$$

Note that this module only calculates the left-hand side of the Euler-Lagrange equations; if we equate the result to zero, we will get the actual geodesic equations. This is hinted at visually by setting the

resulting tensor's symbol to 0, so that you actually see the equations when using **TList**. It is trivial to see that the solution to these equations is simply a curve with a constant velocity; in a flat Minkowski spacetime, particles experience no gravitational force, and thus no acceleration (unless some other force acts on them, of course).

The derivatives with respect to the curve parameter λ are kept unevaluated, using the Mathematica function **Inactive**. This simplifies the equations, and can sometimes help solve them by inspection. If we want to activate the derivatives, we simply need to use **Activate**. Recall that **TList** and **TShow** can apply a function to the tensor's components before displaying them, so we just need to pass **Activate** as the last argument:

```
In[ ]:= TList[TGeodesicFromLagrangian["Minkowski"], Activate]
```

Minkowski→GeodesicFromLagrangian:

$$\begin{aligned} 0_t &= -\ddot{t} \\ 0_x &= \ddot{x} \\ 0_y &= \ddot{y} \\ 0_z &= \ddot{z} \end{aligned}$$

Now the derivatives have been activated; note that all the components have become second derivatives of the coordinates.

As with the Lagrangian itself, the geodesic equations are displayed in compact notation when using **TList**. If we want the full expressions with the explicit derivatives, for example in order to pass them to **DSolve** and actually solve the equations, we can use **TGetComponents**:

```
In[ ]:= TGetComponents[TGeodesicFromLagrangian["Minkowski"]]
```

... **TGetComponents**: Using the default index configuration {-1} and the default coordinate system "Cartesian".

```
Out[ ]:= {D[-t'[λ], λ], D[x'[λ], λ], D[y'[λ], λ], D[z'[λ], λ]}
```

Again, if we wish to activate the derivatives, we need to use **Activate** (recall that a function passed as the last argument to **TGetComponents** is applied to the components before returning them):

```
In[ ]:= TGetComponents[TGeodesicFromLagrangian["Minkowski"], Activate]
```

... **TGetComponents**: Using the default index configuration {-1} and the default coordinate system "Cartesian".

```
Out[ ]:= {-t''[λ], x''[λ], y''[λ], z''[λ]}
```

We can similarly find the geodesic equations of other metrics:

In[*]:= TList@TGeodesicFromLagrangian["Schwarzschild"]

OGRe:

Schwarzschild→GeodesicFromLagrangian:

$$0_t = \partial_\lambda \left(\left(-1 + \frac{2M}{r} \right) \dot{t} \right)$$

$$0_r = \frac{M \dot{r}^2}{(-2M+r)^2} + \frac{M \dot{t}^2}{r^2} - r \left(\dot{\theta}^2 + \dot{\phi}^2 \sin[\theta]^2 \right) + \partial_\lambda \frac{\dot{r}}{1 - \frac{2M}{r}}$$

$$0_\theta = -r^2 \cos[\theta] \dot{\phi}^2 \sin[\theta] + \partial_\lambda (r^2 \dot{\theta})$$

$$0_\phi = \partial_\lambda (r^2 \dot{\phi} \sin[\theta]^2)$$

In[*]:= TList@TGeodesicFromLagrangian["FLRW"]

OGRe:

FLRW→GeodesicFromLagrangian:

$$0_t = -a \partial_t a \left(\dot{r}^2 + r^2 \left(\dot{\theta}^2 + \dot{\phi}^2 \sin[\theta]^2 \right) \right) + \partial_\lambda (-\dot{t})$$

$$0_r = -a^2 r \left(\dot{\theta}^2 + \dot{\phi}^2 \sin[\theta]^2 \right) + \partial_\lambda (a^2 \dot{r})$$

$$0_\theta = -a^2 r^2 \cos[\theta] \dot{\phi}^2 \sin[\theta] + \partial_\lambda (a^2 r^2 \dot{\theta})$$

$$0_\phi = \partial_\lambda (a^2 r^2 \dot{\phi} \sin[\theta]^2)$$

In[*]:= TList@TGeodesicFromLagrangian["Alcubierre"]

OGRe:

Alcubierre→GeodesicFromLagrangian:

$$0_t = -\dot{t} (f v \dot{t} - \dot{z}) (v \partial_t f + f \partial_t v) + \partial_\lambda \left((-1 + f^2 v^2) \dot{t} - f v \dot{z} \right)$$

$$0_x = v \dot{t} (-f v \dot{t} + \dot{z}) \partial_x f + \partial_\lambda \dot{x}$$

$$0_y = v \dot{t} (-f v \dot{t} + \dot{z}) \partial_y f + \partial_\lambda \dot{y}$$

$$0_z = v \dot{t} (-f v \dot{t} + \dot{z}) \partial_z f + \partial_\lambda (-f v \dot{t} + \dot{z})$$

The last example, the geodesic equations of the Alcubierre metric, is a good example of how we can solve the geodesic equations by inspection. Indeed, it is easy to see that

$$\dot{x}^\mu = (1, 0, 0, v f)$$

is a solution to this system of equations, since then we have $\dot{x} = \dot{y} = 0$ and $(f v \dot{t} - \dot{z}) = 0$, and both terms in each equation vanish (the last term in the first equation will reduce to $\partial_\lambda(-1)$, which is of course zero). We can verify this solution by replacing the coordinate functions with their solutions; since we will be left with $\partial_\lambda(-1)$ in the first equation, we must also activate the derivative:

```
In[ ]:= TList[TGeodesicFromLagrangian["Alcubierre"], (Activate@
#1 /. {t'[λ] → 1, x'[λ] → 0, y'[λ] → 0, z'[λ] → v[t[λ]] × f[t[λ], x[λ], y[λ], z[λ]]}) &]
```

OGRe:

Alcubierre→GeodesicFromLagrangian:
No non-zero elements.

Notice how we had to write the coordinates explicitly as **functions of the curve parameter**, even when they are arguments of a function; for example, $v[t]$ became $v[t[\lambda]]$. This solution indicates that we are traveling with velocity v in the z direction; the warp bubble (inside which, as you recall, $f = 1$) moves whatever is inside it, such as a spaceship, through space at the velocity v , but there is no limit on v - it can even be faster than light!

Geodesic equations from the Christoffel symbols

Another way of obtaining the geodesic equations is using the covariant derivative, and thus the Christoffel symbols:

$$\dot{x}^\rho \nabla_\rho \dot{x}^\sigma = 0 \Rightarrow \ddot{x}^\sigma + \Gamma_{\mu\nu}^\sigma \dot{x}^\mu \dot{x}^\nu = 0.$$

In OGRE, we can calculate the left-hand side of this equation using **TGeodesicFromChristoffel**:

```
In[ ]:= ? TGeodesicFromChristoffel
```

Out[]:=

Symbol

TGeodesicFromChristoffel[**metricID**] returns a reference to the geodesic equations obtained using the Christoffel symbols of the metric **metricID**. Equating the components to zero will yield the full system of geodesic equations.

The result will be given in terms of the coordinate symbols as functions of the curve parameter and their derivatives with respect to the curve parameter. The curve parameter can be selected using

TSetCurveParameter[].

▼

For example:

In[]:= TList@TGeodesicFromChristoffel["Minkowski"]

Minkowski→GeodesicFromChristoffel:

$$0^t = \ddot{t}$$

$$0^x = \ddot{x}$$

$$0^y = \ddot{y}$$

$$0^z = \ddot{z}$$

In[]:= TList@TGeodesicFromChristoffel["Schwarzschild"]

Schwarzschild→GeodesicFromChristoffel:

$$0^t = \frac{2M\dot{r}\dot{t}}{r(-2M+r)} + \ddot{t}$$

$$0^r = \frac{M\dot{r}^2}{2Mr-r^2} + \frac{M(-2M+r)\dot{t}^2}{r^3} + \ddot{r} + (2M-r)(\dot{\theta}^2 + \dot{\phi}^2 \sin^2[\theta])$$

$$0^\theta = \frac{2\dot{r}\dot{\theta}}{r} + \ddot{\theta} - \cos[\theta]\dot{\phi}^2 \sin[\theta]$$

$$0^\phi = \frac{2(\dot{r}+r \cot[\theta]\dot{\theta})\dot{\phi}}{r} + \ddot{\phi}$$

In[]:= TList@TGeodesicFromChristoffel["FLRW"]

FLRW→GeodesicFromChristoffel:

$$0^t = \ddot{t} + a \partial_t a (\dot{r}^2 + r^2 (\dot{\theta}^2 + \dot{\phi}^2 \sin^2[\theta]))$$

$$0^r = \ddot{r} + \frac{2\dot{r}\dot{t}\partial_t a}{a} - r(\dot{\theta}^2 + \dot{\phi}^2 \sin^2[\theta])$$

$$0^\theta = \ddot{\theta} + 2\dot{\theta}\left(\frac{\dot{r}}{r} + \frac{\dot{t}\partial_t a}{a}\right) - \cos[\theta]\dot{\phi}^2 \sin[\theta]$$

$$0^\phi = \ddot{\phi} + 2\dot{\phi}\left(\frac{\dot{r}}{r} + \cot[\theta]\dot{\theta} + \frac{\dot{t}\partial_t a}{a}\right)$$

In[]:=

TList@TGeodesicFromChristoffel["Alcubierre"]

Alcubierre->GeodesicFromChristoffel:

$$0^t = \ddot{t} + v(f v \dot{t} - \dot{z})(-\dot{x} \partial_x f - \dot{y} \partial_y f + (f v \dot{t} - \dot{z}) \partial_z f)$$

$$0^x = \ddot{x} + v \dot{t}(-f v \dot{t} + \dot{z}) \partial_x f$$

OGRE:

$$0^y = \ddot{y} + v \dot{t}(-f v \dot{t} + \dot{z}) \partial_y f$$

$$0^z = \ddot{z} - v \dot{t}(\dot{t} \partial_t f + \dot{x} \partial_x f + \dot{y} \partial_y f) + f^3 v^4 \dot{t}^2 \partial_z f - f^2 v^3 \dot{t}(\dot{x} \partial_x f + \dot{y} \partial_y f + 2 \dot{z} \partial_z f) + f(-\dot{t}^2(\partial_t v + v^2 \partial_z f) + v^2 \dot{z}(\dot{x} \partial_x f + \dot{y} \partial_y f + \dot{z} \partial_z f))$$

Often, you will find that the Lagrangian method produces simpler equations, which can even be solved by inspection, as we did for the Alcubierre metric. This is due to the possibility of leaving the λ derivative unevaluated. However, in other cases, the Christoffel method might produce simpler equations. The best thing to do is to try both methods and see which one produces simpler or nicer results for the specific metric in question. Note that the system of equations obtained using `TGeodesicFromLagrangian` will often be different from the one obtained using `TGeodesicFromChristoffel`, but both systems will always have the same solutions.

Changing the curve parameter

By default, the curve parameter is λ . However, sometimes we want to use another parameter - for example τ for proper time. To change the parameter, we use `TSetCurveParameter`:

In[]:=

? TSetCurveParameter

Out[]:=

Symbol

TSetCurveParameter[] shows the curve parameter used for calculating Lagrangians and geodesics.

TSetCurveParameter[**parameter**] changes the curve parameter. The new parameter will be cleared and protected, and the old parameter will be unprotected. **parameter** can be given either as a symbol name or a string representing a symbol name.

TSetCurveParameter[Automatic] resets the curve parameter to the default: λ .

v

Let us change it to τ :

In[]:=

 TSetCurveParameter[τ];

If we get the components of any Lagrangian or affine-parameter geodesic equation we previously calculated, the parameter will now be shown as τ instead of λ :

```
In[ ]:= TGetComponents@TLagrangian["Minkowski"]
```

... **TGetComponents**: Using the default index configuration {} and the default coordinate system "Cartesian".

```
Out[ ]:=
```

$$\{-\mathbf{t}'[\tau]^2 + \mathbf{x}'[\tau]^2 + \mathbf{y}'[\tau]^2 + \mathbf{z}'[\tau]^2\}$$

```
In[ ]:= TList@TGeodesicFromLagrangian["Minkowski"]
```

OGRe:

Minkowski→GeodesicFromLagrangian:

$$0_t = \partial_\tau(-\dot{t})$$

$$0_x = \partial_\tau \dot{x}$$

$$0_y = \partial_\tau \dot{y}$$

$$0_z = \partial_\tau \dot{z}$$

Geodesic equations in terms of the time coordinate

If the metric is a spacetime metric, it is often convenient to obtain the geodesic equations in terms of the time parameter, instead of an affine curve parameter. It can be shown that the geodesic equations in terms of the time coordinate are given by

$$\frac{d^2 x^\sigma}{dt^2} + \left(\Gamma_{\mu\nu}^\sigma - \Gamma_{\mu\nu}^0 \frac{dx^\sigma}{dt} \right) \frac{dx^\mu}{dt} \frac{dx^\nu}{dt} = 0,$$

where we are assuming the time coordinate is t and it is the first (zero) coordinate. These equations can be obtained using **TGeodesicWithTimeParameter**:

```
In[ ]:= ? TGeodesicWithTimeParameter
```

```
Out[ ]:=
```

Symbol

TGeodesicWithTimeParameter[metricID] returns a reference to the geodesic equations obtained using the first coordinate as the parameter. Equating the components to zero will yield the full system of geodesic equations.

The result will be given in terms of the spatial coordinate symbols as functions of the time coordinate and their derivatives with respect to time. The first coordinate of the coordinate system will be assumed to be the time coordinate, even if its symbol is not t .

Note that **TGeodesicWithTimeParameter** assumes time is the first coordinate, but the coordinate does not need to have the symbol t . As an example, the equations for the FLRW metric in terms of a curve parameter are


```
In[ ]:= TList[TGeodesicFromChristoffel["FLRW"], "Cartesian"]
```

FLRW→GeodesicFromChristoffel:

$$0^t = \ddot{t} + a(\dot{x}^2 + \dot{y}^2 + \dot{z}^2) \partial_t a$$

$$0^x = \ddot{x} + \frac{2\dot{t}\dot{x}\partial_t a}{a}$$

$$0^y = \ddot{y} + \frac{2\dot{t}\dot{y}\partial_t a}{a}$$

$$0^z = \ddot{z} + \frac{2\dot{t}\dot{z}\partial_t a}{a}$$

but in terms of t , we only need 3 equations:

```
In[ ]:= TList[TGeodesicWithTimeParameter["FLRW"], "Cartesian"]
```

FLRW→GeodesicWithTimeParameter:

$$0^x = \ddot{x} - \frac{\dot{x}(-2+a^2(\dot{x}^2+\dot{y}^2+\dot{z}^2))\partial_t a}{a}$$

$$0^y = \ddot{y} - \frac{\dot{y}(-2+a^2(\dot{x}^2+\dot{y}^2+\dot{z}^2))\partial_t a}{a}$$

$$0^z = \ddot{z} - \frac{\dot{z}(-2+a^2(\dot{x}^2+\dot{y}^2+\dot{z}^2))\partial_t a}{a}$$

These equations are easier to solve. For simplicity, assume that we are only moving along the x coordinate. Then we only have one equation to solve:

```
In[ ]:= FLRWEq =
TGetComponents[TGeodesicWithTimeParameter["FLRW"], "Cartesian", y'[t]^2 + z'[t]^2 -> 0][[2]]
```

... TGetComponents: Using the default index configuration {1}.

```
Out[ ]:=
```

$$\frac{a'[t] x'[t] (2 - a[t]^2 x'[t]^2)}{a[t]} + x''[t]$$

The solution can be obtained in terms of an integral over $a(t)$:

```
In[ ]:= FLRWSol = DSolve[FLRWEq == 0, x, t]
```

```
Out[ ]:=
```

$$\left\{ \left\{ x \rightarrow \text{Function} \left[\{t\}, c_2 + \int_1^t \frac{1}{\sqrt{a[K[1]]^2 + a[K[1]]^4 c_1}} dK[1] \right] \right\}, \right. \\ \left. \left\{ x \rightarrow \text{Function} \left[\{t\}, c_2 + \int_1^t \frac{1}{\sqrt{a[K[2]]^2 + a[K[2]]^4 c_1}} dK[2] \right] \right\} \right\}$$

From this we can get the coordinate velocity $\dot{x}$ along x :

```

In[ ]:= FullSimplify[a[t] ∂t(x[t] /. FLRWSol[[2]]), a[t] > 0]
Out[ ]:=

$$x'[t] == \frac{1}{a[t] \sqrt{1 + a[t]^2 c_1}}$$


```

Importing and exporting tensors

In a single Mathematica session, one can spend considerable time and effort defining tensors and doing various operations on them. However, as the tensors are only stored in memory, once the session is over and the kernel is stopped, all that information will be lost. Due to the non-linear nature of Mathematica notebooks, even if you saved the entire notebook, it can be hard or even impossible to retrace your steps and get the exact same tensors again from the information in the notebook.

Instead of defining all the tensors from scratch, OGRE allows the user to export tensors and then import them in another session to continue working with them later. The tensors are stored internally as an **Association**, and exporting a tensor essentially amounts to outputting the corresponding **Association**. **Warning: Do not change the exported data manually, as that might break the class invariants and cause errors after importing it back!**

To export a single tensor, use the **TExport** module:

```

In[ ]:= ? TExport
Out[ ]:=
Symbol
TExport[ID] exports the raw tensor data for the tensor object ID as an Association.

```

For example, here is how the 4-velocity is stored internally:

```

In[ ]:= TExport["4-Velocity"]
Out[ ]:=
{4-Velocity → {Components → { { {1}, Cartesian} → { 1/√(1-v²), v/√(1-v²), 0, 0 },
{ {-1}, Cartesian} → { -1/√(1-v²), v/√(1-v²), 0, 0 } },
DefaultCoords → Cartesian, DefaultIndices → {1}, Metric → Minkowski,
Role → Tensor, Symbol → v, OGREVersion → v2.0.0 (2026-05-02) } }

```

This is a nested **Association**. The upper level has just one key: **"4-Velocity"**, which is the ID of the tensor. Its value is another **Association**, which has the following keys:

- **"Components"**: An **Association** containing the components of the tensor in different representations, each with a specific index configuration and coordinate system. The components

are only generated when a particular combination of indices and coordinates is requested for the first time, so for example, here the components with both an upper and a lower index in Minkowski coordinates have been stored, but no components in spherical coordinates, since we have not tried to access them so far.

- **"DefaultCoords"**: The default coordinate system to use when displaying the tensor.
- **"DefaultIndices"**: The default index configuration to use when displaying the tensor.
- **"Metric"**: The unique ID of the metric that will be used to raise and lower the tensor's indices. Note that this is only a reference, so a tensor object with this ID must exist. If a tensor is exported, its metric must be exported separately as well for raising and lowering of indices to work.
- **"Role"**: The role of the tensor, which depends on the module that created it. Will be **"Coordinates"** if the tensor was created using **TNewCoordinates**, **"Metric"** if the tensor was created using **TNewMetric**, or **"Tensor"** if the tensor was created using **TNewTensor**. Other modules that we will discuss below, such as **TCalc**, **TChristoffel**, and **TRiemann**, have corresponding roles as well. Additional roles are only used internally by OGRE, such as **"Temporary"** for a temporary tensor created as an intermediate step in a calculation.
- **"Symbol"**: The symbol used to represent the tensor when displaying it.
- **"OGReVersion"**: The version of the package used to create the tensor. (Note that this key is not stored internally, it is added by **TExport**.)

Other keys also exist in special cases. For example, for tensor objects representing coordinate systems, the keys **"CoordTransformations"** and **"Jacobians"** are used to store the details of coordinate transformations defined using **TAddCoordTransformation**, as can be seen by exporting **"Cartesian"**:

In[]:=
Out[]:=

TExport["Cartesian"]

```
<| Cartesian →
  <| Components → <| {{1}, Cartesian} → {t, x, y, z} |>, DefaultCoords → Cartesian,
  DefaultIndices → {1}, Role → Coordinates, Symbol → x, CoordTransformations →
    <| Spherical → {x → r Cos[φ] Sin[θ], y → r Sin[θ] Sin[φ], z → r Cos[θ]} |>,
  Jacobians → <| Spherical → <| Jacobian →
    {{1, 0, 0, 0}, {0, Cos[φ] Sin[θ], r Cos[θ] Cos[φ], -r Sin[θ] Sin[φ]}, {0,
      Sin[θ] Sin[φ], r Cos[θ] Sin[φ], r Cos[φ] Sin[θ]}, {0, Cos[θ], -r Sin[θ], 0}},
    InverseJacobian → {{1, 0, 0, 0}, {0, Cos[φ] Sin[θ], Sin[θ] Sin[φ], Cos[θ]},
      {0, Cos[θ] Cos[φ], Cos[θ] Sin[φ], -Sin[θ]}, {0, -Csc[θ] Sin[φ],
        Cos[φ] Csc[θ], 0}} |> |>, OGREVersion → v2.0.0 (2026-05-02) |> |>
```

Built-in tensors derived from a metric, such as **TRicci["FLRW"]**, are cached inside the metric, not stored as top-level tensor objects with a unique ID. They can still be exported individually:

In[]:= **TExport**[**TRicci**["FLRW"]]

Out[]=

```

<| FLRW->Ricci -> <| Components ->
  <| {{-1, -1}, Spherical} -> {{- 3 a''[t]
    a[t], 0, 0, 0}, {0, 2 a'[t]^2 + a[t] a''[t], 0, 0}, {0,
    0, r^2 (2 a'[t]^2 + a[t] a''[t]), 0}, {0, 0, 0, r^2 Sin[θ]^2 (2 a'[t]^2 + a[t] a''[t])}}|>,
  DefaultCoords -> Spherical, DefaultIndices -> {-1, -1}, Metric -> FLRW,
  Role -> Ricci, Symbol -> R, OGREVersion -> v2.0.0 (2026-05-02) |>|>

```

In this case, the exported key will have the form "**FLRW->Ricci**"; this is not a tensor ID, but a reference to a cached built-in tensor. The **->** reference operator is reserved for these built-in tensors, and cannot be used in tensor IDs created by the user. Since the built-in tensors are cached inside the metric object, exporting the metric will also export any cached built-in tensors derived from it.

To import a tensor back after it has been exported, use the **TImport** module:

In[]:= **? TImport**

Out[]=

Symbol

TImport[**data**] imports a tensor that has been exported using **TExport**[].

▼

To export **all** of the tensors defined in the current session, we may use the **TExportAll** module:

In[]:= **? TExportAll**

Out[]=

Symbol

TExportAll[] exports the raw tensor data for all tensors defined in the current session as an **Association**.

TExportAll[**filename**] exports the data to **filename**. If a file extension is not provided, the extension **.m** will be appended, unless the file name explicitly ends with a dot. If a full path is not given, the file will be created in the current working directory, as given by **Directory**[] . This directory can be changed using **SetDirectory**[] . Note that the file will be overwritten if it already exists.

▼

The output will be an **Association** as above, where the keys are the names of all the tensors defined so far in the current session, and the value of each key is the data of that tensor. We will not show the complete output here, since it is very long, but let us just demonstrate that the keys of the **Association** are all of the tensors we defined so far in this session:

In[]:=
Out[]=

Keys@TExportAll[]

```
{Options, Cartesian, Spherical, Minkowski, Schwarzschild,
Alcubierre, Kretschmann, PerfectFluid, 4-Velocity, SpatialDistance,
Result, NonSymmetric, PerfectFluidFromVelocity, Position, SimpleMetric,
SimpleMetricManualChristoffel, SimpleMetricManualChristoffelSpherical,
FLRW, PartialDerivatives, SchwarzschildManualRiemann,
SchwarzschildDeSitter, RestVelocity, PerfectFluidFLRW, FLRWConservation}
```

TExportAll exports not only the tensors, but also a special key called **Options**, which contains the current version of the package (for compatibility purposes, because the storage format may change between versions) and any options set by the user during the session. Note that keys associated with tensor objects, such as **"Cartesian"**, **"Spherical"**, and so on, are always strings, but **Options** is not a string; this is to ensure it doesn't get accidentally interpreted as a tensor object. We can see the options configured in this session by reading the value of the **Options** key:

In[]:=
Out[]=

TExportAll[] [Options]

```
<|CurveParameter → Global`τ, ExactSignChecks → True,
IndexLetters → μνρσκλαβγδεζηθιξπτφχψω, OGReVersion → v2.0.0 (2026-05-02),
Parallelize → False, ReservedSymbols → {t, x, y, z, r, θ, φ, M, v, f, ρ, p, a, k, Δ},
SimplifyAssumptions → <|AssumeReal → True, User → {r ≥ 0} |>, SimplifyFunc → FullSimplify|>
```

Note that the settings we have discussed, such as the index letters, reserved symbols, and simplification assumptions, are included. This means that these settings will be imported when you import the tensor data in another Mathematica session - you can start right where you left off, no need to reconfigure anything.

The output of **TExportAll** can be saved in a Mathematica notebook, and imported using the module **TImportAll**:

In[]:=
Out[]=

? TImportAll

Symbol

TImportAll[source] imports tensor data that has been exported using TExportAll[]. If **source** is an Association, imports the data directly. If **source** is a file name, imports the data from that file. If a file extension is not provided, the extension **.m** will be appended, unless the file name explicitly ends with a dot. If a full path is not given, the file is assumed to be located in the current working directory, as given by Directory[]. This directory can be changed using SetDirectory[].

WARNING: This will irreversibly delete ALL of the tensors already defined in the current session.

Note that **TImportAll** will delete any tensors already defined in the current session, whether or not they have the same ID as an imported tensor. To keep them, first export them into an **Association**,

Join it with the **Association** you wish to import, and then use **TImportAll** on the result - or, alternatively, import the tensors one by one using **TImport**. Similarly, any settings configured during the session will be replaced with the imported settings. **TImport** and **TImportAll** will issue a warning when importing data exported by a different OGRE version, and the tensors or settings may not be imported correctly in that case; when transitioning between versions, it is best to re-define all the tensors from scratch.

If a file name is given to **TExportAll**, the output will be saved to that file. If a file extension is not provided, the extension **.m** will be appended, unless the file name explicitly ends with a dot. If only the name of the file is given, and not a full path - e.g. **TExportAll["OGREtensors.m"]** - then the file will be saved in the current working directory, as given by **Directory[]**. To change the working directory, use **SetDirectory[]** before exporting the file. To import from the file, pass the file name to **TImportAll**, e.g. **TImportAll["OGREtensors.m"]**.

Note that if for some reason you would like to delete all the tensors defined so far in the current session, you can simply import an empty **Association** as follows: **TImportAll[<|>]**. Be careful, as this action is **irreversible**!

Lagrangians and affine-parameter geodesic equations are stored internally using the placeholder **\$CurveParam** for the curve parameter. The placeholder is replaced with the user-selected curve parameter only when they are displayed or the components are retrieved. When exporting tensors using **TExport** or **TExportAll**, the tensors will be exported with the placeholder, so that when they are imported later, they will be imported correctly regardless of which curve parameter is currently selected:

In[]:=
Out[]=

```
TExport[TLagrangian["Minkowski"]]
```

```
<| Minkowski->Lagrangian -> <| Components -> <| {{ }, Cartesian} ->
  {-t'[$CurveParam]^2 + x'[$CurveParam]^2 + y'[$CurveParam]^2 + z'[$CurveParam]^2} |>,
  DefaultCoords -> Cartesian, DefaultIndices -> { }, Metric -> Minkowski,
  Role -> Lagrangian, Symbol -> L, OGREVersion -> v2.0.0 (2026-05-02) |> |>
```

TExportAll also exports the choice of user-facing curve parameter separately as part of the **Options** key, and it is imported when using **TImportAll**, so your parameter of choice can persist between sessions.

Parallelization

Improving performance with parallelization

The calculations we have demonstrated so far in this documentation have been quite simple, and should not take more than a second to perform on a decent computer. However, when doing research, calculations can be much more involved, and thus also take more time to complete. Typically, the

most time-consuming part of any tensor calculation is not the tensor operations themselves, but rather the **simplification** of the final result using `FullSimplify`.

If simplification is taking more than a few seconds, it is highly recommended to turn on the **parallelization** feature, which simplifies the components of the tensors in parallel instead of one after the other. This can provide a significant performance boost, proportional to the number of parallel kernels that can be launched. Note that this number is determined by your Mathematica license, and it may be less than the number of cores in your CPU.

Since we would like to measure execution time in this section, let us use the command `ClearSystemCache` to clear Mathematica's cache - since otherwise it will remember the results of previous simplifications, which may artificially speed up the calculation:

```
In[ ]:= ClearSystemCache[]
```

To demonstrate the benefits of parallelization, let us consider the following somewhat complicated non-diagonal metric, which depends on an abstract function f :

```
In[ ]:= TShow[TNewMetric["ParallelizationTest",
    "Cartesian", Table[f[a b t], {a, 1, 4}, {b, 1, 4}]]]
```

OGRe: ParallelizationTest: $g_{\mu\nu}(t, x, y, z) = \begin{pmatrix} f & f[2t] & f[3t] & f[4t] \\ f[2t] & f[4t] & f[6t] & f[8t] \\ f[3t] & f[6t] & f[9t] & f[12t] \\ f[4t] & f[8t] & f[12t] & f[16t] \end{pmatrix}$

We first calculate the Christoffel symbols for this metric **without** parallelization, and use `AbsoluteTiming` to measure how long it takes:

```
In[ ]:= notParallelChristoffel = AbsoluteTiming[TChristoffel["ParallelizationTest"]][[1]]
```

```
Out[ ]:=
```

```
5.71073
```

This is the number of seconds this calculation took on my computer. On your computer this duration may be shorter or longer, but the calculation will invariably take a non-trivial amount of time on any personal computer. The vast majority of that time is spent not on calculating the tensor itself, but on **simplifying** the result.

To turn on parallelization, we use the module `TSetParallelization`:

```
In[ ]:= ? TSetParallelization
Out[ ]=
```

Symbol

TSetParallelization[**True**] enables the parallelization of tensor simplifications, and TSetParallelization[**False**] disables it. The default value is **False**. TSetParallelization[] returns the current value. If simplifications take less than a few seconds, then you should leave parallelization off, as it has a small overhead and may actually impede performance. However, if simplifications are taking more than a few seconds, then it is highly recommended to enable parallelization for a significant performance boost.

Let us turn it on now:

```
In[ ]:= TSetParallelization[True]
```

```
OGRe: Parallelization enabled.
```

```
OGRe: 16 parallel kernels launched. CPU has 24 cores.
```

When parallelization is first turned on, all available parallel kernels are automatically launched. As you can see, my system has a 24-core CPU, but only 16 parallel kernels can be launched, since that is what my Mathematica license allows.

We will now repeat the calculation of the Christoffel symbols, in order to see how its performance improves with parallelization. However, before we can do that, we must again use the command **ClearSystemCache** to clear Mathematica's cache:

```
In[ ]:= ClearSystemCache[]
```

(If you don't trust that **ClearSystemCache** is enough to make a reliable benchmark, you can exit the kernel using **Quit**, and then reload the package and redefine the Cartesian coordinates and the test metric.)

We will also delete the tensor that we calculated previously, so that we can calculate it again (if overwriting is not turned on):

```
In[ ]:= TDelete[TChristoffel["ParallelizationTest"]];
```

Now we can accurately measure the execution time for **TChristoffel** with parallelization:

```
In[ ]:= parallelChristoffel = AbsoluteTiming[TChristoffel["ParallelizationTest"]][[1]]
Out[ ]:=
```

```
1.56623
```

We see that the calculation now took less time, with an improvement by a factor of:


```
In[ ]:=
notParallelChristoffel
parallelChristoffel

Out[ ]=
```

```
3.64617
```

With longer calculations, the improvement will be even more significant; I intentionally chose an example that doesn't take too long to calculate, because I evaluate this notebook on a regular basis as part of the package's development. Increasing the number of kernels (if it was allowed by my license) would provide an additional speedup.

As a rule of thumb, if simplifications are taking less than a few seconds, then you should leave parallelization off, as it has a small overhead and may actually impede performance in that case. However, if simplifications are taking more than a few seconds, then it is highly recommended to enable parallelization for a significant performance boost.

Making use of OGRe's parallelized simplifications for other Mathematica expressions

When **TSimplify** is applied to any expression which is not a tensor object, the object is simplified based on the user-defined simplification assumptions set using **TSetAssumptions**:

```
In[ ]:=
TSetAssumptions [ ]

Out[ ]=
```

```
<| AssumeReal → True, User → { r ≥ 0 } |>
```

```
In[ ]:=
TSimplify[ $\sqrt{r^2}$ ]

Out[ ]=
```

```
r
```

If the expression is a **List**, the components will be simplified in parallel. The user can thus make use of OGRe's optimized simplification process to simplify any Mathematica expression.

About the project

Bug reports and feature requests

This package is under continuous and active development. If you encounter any bugs, or if you would like to request any additional features, please feel free to open a new issue on GitHub and I will look into it as soon as I can.

Contribution and pull request policy

Contributions are always welcome. However, I release my projects in cumulative updates after editing and testing them locally on my system, so **my policy is to never accept any pull requests**. If you open a pull request, and I decide to incorporate your suggestion into the project, I will first modify your code to comply with the project's coding conventions (formatting, syntax, naming, comments, programming practices, etc.), and perform some tests to ensure that the change doesn't break anything. I will then merge it into the next release of the project, possibly together with some other changes. The new release will also include a note in `CHANGELOG.md` with a link to your pull request, and modifications to the documentation in `README.md` as needed.

Starring the repository

If you found this project useful, please consider starring it on GitHub! This allows me to see how many people are using my code, and motivates me to keep working to improve it.

Acknowledgements

A major portion of the code for this package was written while I was a postdoctoral researcher in Niayesh Afshordi's group at Perimeter Institute for Theoretical Physics in Waterloo, Ontario, Canada. I would like to thank Niayesh Afshordi and Perimeter Institute for their support during that time. Research at Perimeter Institute is supported in part by the Government of Canada through the Department of Innovation, Science, and Economic Development Canada and by the Province of Ontario through the Ministry of Colleges and Universities.

Further work on this package was done as an assistant professor at Brock University. I acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC), RGPIN-2024-04063.

Copyright and citing

Copyright (c) 2021-2026 Barak Shoshany. Licensed under the MIT license.

If you use this package in software of any kind, please provide a link to the GitHub repository in the source code and documentation.

If you use this package in published research, please cite it as follows:

- Shoshany, B., (2021). OGRE: An Object-Oriented General Relativity Package for Mathematica. Journal of Open Source Software, 6(65), 3416, <https://doi.org/10.21105/joss.03416>

You can also use the following BibTeX entry:

```
@article{Shoshany2021_OGRE,
  author    = {Barak Shoshany},
  doi       = {10.21105/joss.03416},
  journal   = {Journal of Open Source Software},
  number    = {65},
```

```

pages      = {3416},
publisher  = {The Open Journal},
title      = {{OGRe}: An Object-Oriented General Relativity Package for Mathematica},
url        = {https://doi.org/10.21105/joss.03416},
volume     = {6},
year       = {2021}
}

```

Citing information can always be obtained by executing the command `TCite[]`:

In[]:=

?TCite

Out[]:=

Symbol

TCite[] displays information on how to cite this package in published research. Thank you for citing my work! :)

▼

Other projects to check out

This package has a Python port, `OGRePy: An Object-Oriented General Relativity Package for Python`. If you are interested in doing OGRE-style tensor calculations in Python, please check it out!

If you are interested in high-performance scientific computing with C++, and want to get the maximum performance possible from multi-core CPUs, please check out `BS::thread_pool`: a fast, lightweight, modern, and easy-to-use C++17 / C++20 / C++23 thread pool library.